

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
“ВЫСШАЯ ШКОЛА ЭКОНОМИКИ”**

Факультет компьютерных наук
Образовательная программа “Программная инженерия”

СОГЛАСОВАНО

Научный руководитель
Инженер-разработчик, Т-Банк
Приглашённый преподаватель
департамента программной
инженерии

_____/ Е.А. Каравасева /
“ ” _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»
Старший преподаватель департамента
программной инженерии

_____/ Н.А. Павлов /
“ ” _____ 2026 г.

**ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ ОНЛАЙН-ОБУЧЕНИЯ
С ПОДДЕРЖКОЙ ВИДЕОСВЯЗИ
И ИИ-ПОМОЩНИКА.
СЕРВЕРНАЯ ЧАСТЬ 1**

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.12.17-01 81 01–3-ЛУ

Исполнитель:
Студент группы БПИ244

_____/ С.М. Павлычев /
“ ” _____ 2026 г.

Москва 2026

Изн.№ подл.	Подп. и дата	Взам. инв.№	Изн.№ дубл.	Подп. и дата

УТВЕРЖДЁН
RU.17701729.12.17-01 81 01–3-ЛУ
11.05.2026

**ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ ОНЛАЙН-ОБУЧЕНИЯ
С ПОДДЕРЖКОЙ ВИДЕОСВЯЗИ
И ИИ-ПОМОЩНИКА.
СЕРВЕРНАЯ ЧАСТЬ 1**

Пояснительная записка

RU.17701729.12.17-01 81 01–3

Листов 48

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Настоящий документ «Пояснительная записка» разработан в соответствии с ГОСТ 19.404–79 и содержит обоснование технических решений, принятых при разработке серверной части 1 программного изделия «Веб-приложение для онлайн-обучения с поддержкой видеосвязи и ИИ-помощника» (далее – Платформа, краткое наименование – «Профессия»), в зоне ответственности исполнителя С.М. Павлычева.

Документ описывает назначение и область применения разработанных программных компонентов, алгоритмы функционирования серверных приложений: «courses» (динамическое построение иерархии материалов, многоуровневое кеширование каталога, ролевая модель доступа), «homeworks» (жизненный цикл попытки, гибридная система скоринга, атомарная обработка черновиков), «payments» (идемпотентная обработка транзакций через подменный сервис, асинхронные цепочки зачислений), «notifications» (маршрутизация событий по приоритетам, потоковая доставка через SSE на базе брокера сообщений, интеграция с Notifcore), «ai_chat_bot» (реализация метода RAG, управление контекстным окном, потоковая генерация ответов), «admin_panel» (управление токенизированными приглашениями), «applications» (автоматизация воронки зачислений на спецкурсы) и «core» (управление жизненным циклом ресурсов, механизм Presigned URLs, сборка «мусора» на основе счётчика ссылок).

Обоснован выбор технологического стека (Python 3.14, Django 6, Celery, Redis, PostgreSQL 15), обеспечивающего высокую доступность и консистентность данных. Содержит технико-экономические показатели и сравнение с аналогами.

Составлен в соответствии с требованиями ГОСТ 19.105–78 и ГОСТ 19.404–79. Изменения должны быть оформлены согласно ГОСТ 19.603–78 и ГОСТ 19.604–78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

АННОТАЦИЯ	2
1. ВВЕДЕНИЕ	5
1.1. Наименование программы	5
1.2. Краткая характеристика области применения	5
2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ	6
2.1. Назначение программы	6
2.2. Область применения программы	7
2.3. Категории конечных потребителей в зоне ответственности	7
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	9
3.1. Постановка задачи на разработку	9
3.1.1. Описание задачи	9
3.1.2. Описание применяемых методов	9
3.1.3. Ограничения и допущения	11
3.2. Описание алгоритмов и функционирования программы	11
3.2.1. Подсистема «courses» – каталог и содержимое курсов	12
3.2.2. Подсистема «homeworks» – домашние задания и попытки	12
3.2.3. Подсистема «payments» – платёжный цикл	13
3.2.4. Подсистема «notifications» – уведомления	13
3.2.5. Подсистема «ai_chat_bot» – помощник на основе языковой модели	14
3.2.6. Подсистема «admin_panel» – панель модератора	14
3.2.7. Подсистема «applications» – заявки на специальные курсы	15
3.2.8. Подсистема «core» – медиафайлы	15
3.3. Описание инфраструктурных компонентов и их параметров	16
3.3.1. Система управления базами данных PostgreSQL	16
3.3.2. Кеш на основе хранилища оперативной памяти Redis	16
3.3.3. Брокер сообщений RabbitMQ	16
3.3.4. Сервер обработки запросов Daphne	16
3.3.5. Объектное хранилище Yandex Object Storage	17
3.4. Описание и обоснование выбора метода организации входных и выходных данных	17
3.4.1. Входные данные	17
3.4.2. Выходные данные	18
3.5. Описание и обоснование выбора состава технических и программных средств	18
3.5.1. Язык программирования Python версии 3.14	18
3.5.2. Веб-фреймворк Django и расширение для программного интерфейса	18
3.5.3. Система фоновых задач Celery и брокер RabbitMQ	19
3.5.4. Хранилище оперативной памяти Redis	19
3.5.5. Система управления базами данных PostgreSQL	19
3.5.6. Языковая модель Yandex GPT и хранилище Yandex Vector Store	19
3.5.7. Объектное хранилище Yandex Object Storage	19

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.6. Взаимодействие с другими программами	19
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	21
4.1. Показатели производительности	21
4.2. Показатели надёжности	22
4.3. Показатели ресурсоёмкости	22
4.4. Экономические преимущества разработанного решения	23
4.4.1. Автоматизация проверки домашних заданий	23
4.4.2. Надёжный платёжный цикл с повторными попытками при сбое	23
4.4.3. Снижение нагрузки на поддержку благодаря контекстному помощнику	24
4.4.4. Сохранение успеваемости через напоминания о сроках сдачи	24
4.5. Ожидаемая потребность	24
4.6. Сравнение с отечественными и зарубежными аналогами	24
5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ	26
5.1. Стандарты Единой системы программной документации	26
5.2. Документация веб-фреймворка и инфраструктуры серверной части	26
5.3. Документация внешних сервисов	27
5.4. Стандарты и спецификации	27
5.5. Прочие источники	28
ПРИЛОЖЕНИЕ 1 – ОПИСАНИЕ ВНЕШНИХ ВЗАИМОДЕЙСТВИЙ КОМПОНЕНТОВ ...	29
Взаимодействие с клиентской частью	29
Взаимодействие с инфраструктурными компонентами	30
Взаимодействие с внешними сервисами	30
ПРИЛОЖЕНИЕ 2 – ПЕРСОНАЛЬНЫЙ ВКЛАД И ГРАНИЦЫ ОТВЕТСТВЕННОСТИ	32
Ключевые проектные решения исполнителя	33
ПРИЛОЖЕНИЕ 3 – ГЛОССАРИЙ И ПЕРЕЧЕНЬ СОКРАЩЕНИЙ	34
ПРИЛОЖЕНИЕ 4 – ОПИСАНИЕ БАЗЫ ДАННЫХ	39
Фрагменты ER-диаграммы, Кардинальность связей и бизнес-логика	39
Таблицы без отдельного фрагмента ER на рисунках П4.1–П4.7	46
Сводная таблица таблиц базы данных в зоне ответственности	47

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «Веб-приложение для онлайн-обучения с поддержкой видеосвязи и ИИ-помощника».

Наименование программы на английском языке – «Web Application for Online Learning with Video Conferencing and AI Assistant».

Краткое наименование программы – «Профессия».

Настоящая пояснительная записка описывает серверную часть 1 Платформы, разработанную в зоне ответственности исполнителя С.М. Павлычева. Обозначение документа по ГОСТ 19.103-77: RU.17701729.12.17-01 81 01–3.

1.2. Краткая характеристика области применения

В настоящее время наблюдается устойчивый и стабильно растущий спрос на дистанционное образование. Всё больше учащихся выбирают онлайн-формат как полноценную альтернативу или дополнение к очному обучению, а работодатели рассматривают цифровые курсы повышения квалификации как регулярную и обязательную форму профессионального развития сотрудников. При этом большинство массовых образовательных платформ решают лишь задачу предоставления видеолекций, однако значительно реже обеспечивают единое рабочее пространство, в котором онлайн-встречи с преподавателем, интерактивная доска, система проверки домашних заданий и контекстный помощник на основе языковой модели доступны в рамках одного согласованного учебного процесса.

Целевой аудиторией разрабатываемой платформы «Профессия» являются учащиеся, заинтересованные в систематизированном изучении предмета под руководством преподавателя: школьники старших классов, студенты средних и высших учебных заведений, а также взрослые слушатели курсов повышения квалификации. Дополнительно платформой пользуются преподаватели и авторы курсов, для которых она выступает единым рабочим инструментом подготовки материалов, проведения вебинаров и проверки учебных работ.

Областью применения программы является сфера цифровых технологий и дистанционного образования.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Назначение программы

Веб-приложение «Профессия» предназначено для централизованного предоставления услуг онлайн-обучения: размещения каталога платных и специальных курсов, публикации учебных материалов в иерархической структуре, выдачи и проверки домашних заданий, организации платёжного цикла, доставки уведомлений в режиме реального времени и контекстной помощи на основе языковой модели.

Настоящая пояснительная записка обосновывает технические решения, принятые исполнителем С.М. Павлычевым. Зона ответственности охватывает следующие программные компоненты.

1. **Серверное приложение «courses»** – ведение полной иерархии учебных материалов (курс – разделы – уроки – домашние задания – вопросы и задачи) с поддержкой ролевого доступа, публичного лендинга с кешированием, управления зачислением на курс и формирования расписания занятий пользователя.
2. **Серверное приложение «homeworks»** – атомарное создание и обработка попыток выполнения домашних заданий: автоматическая проверка вопросов с вариантами ответа, ручная проверка задач с развёрнутым ответом преподавателем, уведомление студента об итогах проверки.
3. **Серверное приложение «payments»** – платёжный цикл через подменный сервис: создание платёжной записи, асинхронная обработка с автоматическими повторами при сбое, создание записи о зачислении на курс при успешной оплате, хранение истории платежей пользователя.
4. **Серверное приложение «notifications»** – единая внутренняя шина событий приложения, многоканальная доставка уведомлений: внутривыплатформенный список с постраничной прокруткой, потоковая доставка событий в режиме реального времени через серверный поток событий поверх брокера сообщений, отправка по электронной почте, отправка по SMS через сервис «Notifcore».
5. **Серверное приложение «ai_chat_bot»** – помощник на основе языковой модели Yandex GPT: постоянное двустороннее соединение, управление сессиями и чатами пользователя, потоковая передача ответов частями по мере их генерации, синхронизация содержимого курса с векторным хранилищем методом дополнения поиском.
6. **Серверное приложение «admin_panel»** – панель модератора: управление составом преподавателей, выдача одноразовых токенов-приглашений со сроком действия 3 суток, назначение и снятие авторов курса, публикация и снятие с публикации курсов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

7. **Серверное приложение «applications»** – заявки студентов на зачисление в специальные курсы без оплаты: подача, рассмотрение (одобрение или отклонение), автоматическое зачисление при одобрении.
8. **Серверное приложение «core»** – подсистема медиафайлов: выдача предподписанных ссылок для прямой загрузки файлов в объектное хранилище, отслеживание статуса готовности файла, периодическая очистка зависших и неиспользуемых файлов.

2.2. Область применения программы

Областью применения программы является сфера дистанционного образования. Платформа ориентирована на круглогодичное использование без выраженных сезонных всплесков нагрузки.

Целевыми пользовательскими сегментами программы являются:

- учащиеся школьного, среднего и высшего профессионального образования, а также взрослые слушатели курсов повышения квалификации, заинтересованные в систематизированном изучении предмета в дистанционном формате;
- преподаватели и авторы курсов, использующие платформу как единое рабочее пространство для публикации материалов, проверки домашних заданий и наблюдения за прогрессом учеников;
- модераторы и администраторы образовательной программы, отвечающие за приглашение преподавателей, публикацию курсов и общий контроль платформы.

2.3. Категории конечных потребителей в зоне ответственности

В таблице 2.1 приведено распределение возможностей каждой подсистемы по категориям пользователей.

Таблица 2.1 – Категории пользователей и подсистемы в зоне ответственности

Подсистема	Студент	Преподаватель	Модератор
«courses» – каталог и структура курсов	Просмотр опубликованных курсов и материалов при наличии зачисления	Создание и редактирование курсов, разделов, уроков, домашних заданий	Публикация и снятие курсов с публикации, назначение авторов
«homeworks» – домашние задания	Создание и отправка попыток, просмотр результатов проверки	Просмотр попыток студентов, выставление баллов за задачи с развёрнутым ответом	Не используется в роли модератора
«payments» – платежи	Оплата курсов, просмотр истории собственных платежей	Не используется в роли преподавателя	Не используется в роли модератора

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Подсистема	Студент	Преподаватель	Модератор
«notifications» – уведомления	Получение уведомлений об оценке, дедлайнах, новых уроках	Получение уведомлений об активности по своим курсам	Получение системных уведомлений
«ai_chat_bot» – ии-ассистент	Отправка вопросов по материалам зачисленного курса, получение ответов	Не используется в роли преподавателя	Не используется в роли модератора
«admin_panel» – панель модератора	Не используется в роли студента	Просмотр списка своих приглашений	Управление преподавателями, приглашениями и курсами
«applications» – заявки на специальные курсы	Подача заявки на зачисление без оплаты	Одобрение или отклонение заявок по своим курсам	Одобрение или отклонение заявок на любой курс
«core» – сервис по управлению внутренним медиа	Загрузка файлов к домашним заданиям	Загрузка медиаматериалов для уроков	Не используется напрямую

Подробное распределение прав доступа на уровне отдельных адресов программного интерфейса соответствует ролевой модели, описанной в приложении 2 общего технического задания (RU.17701729.12.17-01 ТЗ 01-3).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку

3.1.1. Описание задачи

Требуется разработать серверную часть 1 веб-приложения «Профессия» – комплекс из восьми программных модулей («courses», «homeworks», «payments», «notifications», «ai_chat_bot», «admin_panel», «applications», «core»), реализованных в виде приложений фреймворка Django, которые совместно обеспечивают следующие функции.

1. Управление иерархией учебных курсов с поддержкой разграничения прав по роли пользователя, автоматическим кешированием публичного каталога в хранилище оперативной памяти и управлением зачислением студентов.
2. Создание, отправка и проверка домашних заданий двух типов: вопросы с вариантами ответа (автоматическая проверка без участия преподавателя) и задачи с развёрнутым ответом (ручная проверка преподавателем).
3. Реализация платёжного цикла через подменный сервис с асинхронной фоновой обработкой и записью результата зачисления.
4. Многоканальная доставка уведомлений: список внутри платформы, потоковая доставка через серверный поток событий поверх брокера сообщений, электронная почта, SMS.
5. Канал взаимодействия с помощником на основе языковой модели Yandex GPT с синхронизацией содержимого курса в векторном хранилище методом дополнения поиском, с потоковой передачей ответа по мере генерации.
6. Административные инструменты управления преподавателями, приглашениями, публикацией курсов.
7. Обработка заявок студентов на специальные курсы без платёжного цикла.
8. Подсистема медиафайлов: выдача предподписанных ссылок, отслеживание статуса, периодическая очистка.
9. Предоставление программного интерфейса в формате обмена данными JSON с описанием в формате OpenAPI версии 3.0.

3.1.2. Описание применяемых методов

Метод иерархической модели данных с автоматическим присвоением порядковых номеров. Структура курса организована в виде четырёхуровневой иерархии: «Курс» – «Раздел» – «Урок» – «Домашнее задание». Порядковые номера разделов и уроков присваиваются автоматически в пределах родительского элемента через механизм сигналов предварительного сохранения записи

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

в базу данных. Это исключает необходимость ручного управления нумерацией и гарантирует корректный порядок отображения при любых операциях добавления и удаления элементов.

Метод серверного кеширования публичных данных. Публичный каталог курсов сохраняется в кеше на основе хранилища оперативной памяти со временем жизни не менее 600 секунд. Сброс кеша выполняется автоматически через сигналы сохранения и удаления записи при любом изменении курса – вне зависимости от источника изменения (программный интерфейс, панель управления, прямое обращение к базе данных через слой объектно-реляционного отображения). Это обеспечивает согласованность кеша и снижает нагрузку на базу данных при высокой частоте обращений к каталогу.

Метод подстановки ссылок на медиаресурсы при сохранении урока. Содержимое урока хранится в виде структурированного документа в формате JSON. При сохранении клиент сохраняет медиафайлы в S3 хранилище используя временные ссылки, которые ему выдает сервер. При получении урока клиент заменяет ссылки на конкретные объекты из S3.

Метод атомарного создания черновика попытки. Черновик попытки выполнения домашнего задания создаётся атомарно с использованием операции «получить или создать» на уровне базы данных. Уникальность записи гарантируется составным ограничением уникальности на паре «студент – домашнее задание». Это исключает появление дублирующих попыток при параллельных запросах и гарантирует идемпотентность операции – то есть её результат не изменяется при повторных вызовах.

Метод двухэтапной проверки домашнего задания. При отправке попытки система немедленно проверяет вопросы с заранее заданным правильным ответом и вычисляет промежуточный балл. Задачи с развёрнутым ответом остаются в статусе «ожидает проверки» и оцениваются преподавателем вручную. Разделение автоматической и ручной проверки позволяет студенту сразу получить частичный результат без необходимости ждать преподавателя.

Метод асинхронной обработки платежей с повторными попытками при сбое. Обработка платежа выполняется фоновой задачей очереди с числом повторных попыток до 5 и задержкой 15 секунд между ними. Подменный платёжный сервис имитирует реальный платёжный шлюз с долей успешных платежей около 80 %. При успешной оплате создаётся запись о зачислении со сроком действия согласно политике курса. Такая схема обеспечивает устойчивость к временной недоступности платёжного сервиса и позволяет заменить подменный сервис на реальный без изменения бизнес-логики.

Метод публикации событий через единую шину уведомлений. Все события, требующие уведомления пользователя (новое домашнее задание, изменение срока сдачи, результат проверки, начало занятия), направляются через единый диспетчер событий. Диспетчер маршрутизирует событие во все настроенные каналы доставки: внутренний список, серверный поток событий, электронная почта, SMS. Это исключает дублирование логики уведомлений в разных частях программы и позволяет добавлять новые каналы без изменения основной бизнес-логики.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Метод потоковой доставки уведомлений с подпиской через брокер сообщений. Эндпоинт серверного потока событий подписывается на тематический обменник брокера сообщений с маршрутами по идентификатору пользователя, курса, вебинара и системному каналу. При обрыве соединения серверная часть корректно завершает поток; при повторном подключении клиент получает актуальное состояние очереди. Каждые 30 секунд отправляется служебный сигнал для удержания соединения через промежуточные прокси-серверы.

Метод дополнения поиском для контекстного помощника. При создании или обновлении курса его содержимое асинхронно загружается в векторное хранилище. При обработке вопроса студента система выполняет семантический поиск по базе знаний и передаёт наиболее релевантные фрагменты материалов в языковую модель в качестве дополнительного контекста. Это ограничивает ответы помощника тематикой конкретного курса и повышает их точность.

Метод управления жизненным циклом медиафайлов. Каждый загружаемый файл регистрируется в базе данных в статусе «ожидает загрузки». Переход в статус «готов» происходит после подтверждения клиентом или по событию от хранилища. Периодические фоновые задачи удаляют зависшие файлы (время жизни 24 часа) и неиспользуемые файлы (отсрочка 7 суток). Счётчик ссылок на файл защищает от преждевременного удаления файлов, на которые ссылаются другие сущности системы.

3.1.3. Ограничения и допущения

1. В качестве платёжного шлюза используется учебный подменный сервис с программируемым процентом успешных платежей около 80 %. Интеграция с реальным провайдером оставлена за рамками текущей версии по согласованию с заказчиком.
2. Ответы помощника формируются на основе материалов конкретного курса. Запросы, выходящие за пределы тематики курса, а также нецензурные и оскорбительные обращения, отклоняются системой модерации.
3. Ролевая модель включает три роли: «студент», «преподаватель», «модератор». Переключение ролей в текущей версии не поддерживается.
4. Максимальный размер загружаемого документа через подсистему медиафайлов составляет 1 ГБ. Видеоматериалы загружаются непосредственно во внешний видеохостинг Kinescope и данным ограничением не охватываются.
5. Содержимое урока хранится в виде JSON-документа фиксированной схемы, согласованной с клиентской частью. Изменение схемы требует выполнения миграции данных.

3.2. Описание алгоритмов и функционирования программы

В настоящем подразделе описаны внутренняя структура и алгоритмы работы каждой из реализованных подсистем серверной части. Все подсистемы реализованы как отдельные

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

приложения фреймворка Django в директории «backend/apps/» репозитория проекта и взаимодействуют между собой через прямые вызовы программных модулей на языке Python.

3.2.1. Подсистема «courses» – каталог и содержимое курсов

Структура данных. Иерархия строится из четырёх уровней: объект «Курс» содержит объекты «Раздел», каждый раздел содержит объекты «Урок», каждый урок содержит объекты «Домашнее задание». Домашнее задание включает «Вопросы» (с вариантами ответа, проверяются автоматически) и «Задачи» (с развёрнутым ответом, проверяются преподавателем). Связь «студент – курс» фиксируется в записи о зачислении с полями: источник зачисления (оплата или одобренная заявка), срок действия доступа и признак активности.

Алгоритм публичного каталога с кешированием. Эндпоинт получения списка опубликованных курсов при первом запросе сохраняет результат в кеше с ключом и временем жизни 600 секунд. Последующие запросы обслуживаются из кеша без обращения к базе данных. При любом изменении курса сигнал сохранения записи сбрасывает ключ кеша.

Алгоритм контроля доступа к материалам урока. При запросе содержимого урока сервер проверяет наличие активной записи о зачислении для пары «пользователь – курс»: запись должна быть активна, а срок действия – не истёк. При нарушении любого из условий сервер возвращает ответ об отказе в доступе. Это гарантирует, что материалы доступны только в пределах оплаченного или одобренного периода.

Алгоритм обработки ссылок в содержимом урока. В поле «document» таблицы урока хранится JSON с идентификаторами медиа в нотации «local»:

Алгоритм планирования напоминаний о сроке сдачи. При создании или изменении домашнего задания (включая изменение срока сдачи) система отменяет ранее запланированные напоминания и ставит новые фоновые задачи: напоминание за 24 часа до срока и напоминание за 1 час до срока. При удалении домашнего задания все связанные напоминания отменяются.

3.2.2. Подсистема «homeworks» – домашние задания и попытки

Структура данных. Запись «Попытка» хранит ссылки на студента и домашнее задание, статус (черновик, отправлена, проверена), дату отправки, итоговый балл и проверяющего преподавателя. К попытке прикреплены «Ответы на вопросы» (с признаком корректности) и «Ответы на задачи» (с прикрепленными файлами). К каждому ответу на задачу привязывается «Оценка преподавателя» с баллом и комментарием.

Алгоритм создания черновика. Сервис атомарно выполняет операцию «получить или создать» для пары «студент – домашнее задание». Существующий черновик возвращается без изменений. Повторный перевод попытки в черновик из других статусов не допускается.

Алгоритм отправки попытки. Сервис последовательно выполняет следующие шаги:

1. Проверяет соответствие идентификатора попытки и принадлежности студента.
2. Проверяет статус попытки: допускается отправка только из статуса «черновик».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. Проверяет готовность всех прикрепленных медиафайлов: все должны иметь статус «готов».
4. Запускает автоматическую проверку вопросов с вариантами: для каждого ответа сравнивает выбранный вариант с эталонным, выставляет признак корректности и суммирует баллы.
5. Вычисляет итоговый балл за вопросы.
6. Переводит попытку в статус «отправлена».

Алгоритм ручной проверки задачи. Сервис принимает балл и комментарий (не более 1500 символов) для конкретного ответа на задачу. После выставления оценок за все задачи пересчитывает итоговый балл (сумма баллов за вопросы и задачи) и переводит попытку в статус «проверена». Через диспетчер событий публикуется событие проверки, уведомляющее студента.

3.2.3. Подсистема «payments» – платёжный цикл

Структура данных. Запись «Платёж» хранит общую сумму, статус (ожидает, одобрен, успешен, не удался, возвращён), идентификатор во внешнем сервисе и ссылку на форму оплаты. К платежу привязаны записи «Позиция платежа» (курс и цена на момент покупки). Составное ограничение уникальности на паре «платёж – курс» исключает дублирование позиций.

Алгоритм инициирования оплаты. Эндпоинт принятия оплаты корзины создаёт запись «Платёж» в статусе «ожидает», формирует записи позиций для каждого курса из корзины, обращается к подменному сервису для получения идентификатора и ссылки на форму оплаты, затем помещает в очередь фоновую задачу обработки платежа.

Алгоритм асинхронной обработки. Фоновая задача опрашивает подменный сервис о статусе платежа. Сервис возвращает «успех» с вероятностью около 80 % и «отказ» в остальных случаях. При успехе задача переводит платёж в статус «успешен» и создаёт записи о зачислении на каждый курс из позиций платежа с источником «оплата» и сроком действия согласно политике курса. При отказе – переводит в статус «не удался». При временном сбое задача повторяется до 5 раз с задержкой 15 секунд.

3.2.4. Подсистема «notifications» – уведомления

Структура данных. Запись «Уведомление» хранит тип (персональное, по курсу, системное), текст, признак прочтения и список прочитавших пользователей через связь «многие ко многим».

Алгоритм потокового соединения. Эндпоинт серверного потока событий открывает постоянное соединение. Сервер подписывается на тематический обменник брокера сообщений с маршрутами для конкретного пользователя, его курсов, вебинаров и системного канала. При поступлении сообщения оно немедленно пересылается клиенту. Каждые 30 секунд отправляется служебный сигнал для поддержания соединения. При сбое подключения к брокеру сообщений выполняется до 5 повторных попыток с нарастающей задержкой.

Алгоритм диспетчера событий. Класс диспетчера принимает типизированное событие (например, «Новое домашнее задание», «Изменение срока сдачи», «Результат проверки»,

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

«Запланировано занятие») и маршрутизирует его во все настроенные каналы: создаёт запись уведомления в базе данных, публикует событие в брокер сообщений для потокового канала, отправляет письмо по электронной почте и при необходимости SMS.

3.2.5. Подсистема «ai_chat_bot» – помощник на основе языковой модели

Структура данных. Запись «Сессия» объединяет пользователя и курс, содержит набор чатов. Запись «Чат» хранит тему и упорядоченный список сообщений (роль отправителя – пользователь или помощник, текст, не более 1000 символов).

Алгоритм постоянного соединения. Обработчик соединений при подключении клиента к двустороннему каналу проверяет токен аутентификации через специализированный промежуточный слой. При успешном подключении отправляет клиенту начальное сообщение с перечнем существующих чатов сессии.

Поддерживаемые входящие команды от клиента:

- «начать новый чат» – создаёт новый чат, возвращает подтверждение;
- «удалить чат» – удаляет чат, возвращает подтверждение;
- «получить историю» – возвращает историю сообщений выбранного чата;
- «отправить сообщение» – инициирует генерацию ответа.

Алгоритм потоковой генерации ответа. При получении команды отправки сообщения система сохраняет текст вопроса и отправляет клиенту сигнал начала генерации. Сервис обращения к языковой модели вызывает Yandex GPT в режиме потоковой передачи. Каждый полученный фрагмент ответа немедленно передаётся клиенту отдельным сообщением. По завершении генерации отправляется сигнал окончания и сохраняется полный текст ответа в историю чата. Механизм контроля одновременности ограничивает число параллельных обращений к языковой модели для предотвращения превышения установленных квот.

Алгоритм синхронизации базы знаний. При создании или обновлении курса запускается фоновая задача, которая обходит все уроки, извлекает текстовое содержимое, формирует векторные представления и загружает их в Yandex Vector Store. При обработке вопроса студента система выполняет семантический поиск по базе знаний курса и передаёт наиболее релевантные фрагменты как дополнительный контекст в запрос к языковой модели.

3.2.6. Подсистема «admin_panel» – панель модератора

Структура данных. Запись «Приглашение» хранит одноразовый токен длиной 64 символа, срок действия (3 суток), статус (активно, использовано, просрочено) и ссылку на модератора-отправителя.

Алгоритм приглашения преподавателя. Эндпоинт отправки приглашения, доступный только модератору, создаёт запись с уникальным токеном, сроком действия 3 суток и статусом «активно», затем отправляет письмо с ссылкой для регистрации. Периодическая фоновая задача

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

автоматически переводит истёкшие приглашения в статус «просрочено». После использования ссылки токен переходит в статус «использовано» и становится недействительным.

Управление авторами и публикацией. Эндпоинты добавления и удаления автора управляют составом авторов курса. Эндпоинты публикации и снятия с публикации изменяют флаг видимости курса; при публикации автоматически выполняется сброс кеша каталога.

3.2.7. Подсистема «applications» – заявки на специальные курсы

Структура данных. Запись «Заявка» хранит ссылку на студента и курс, статус (на рассмотрении, одобрена, отклонена), дату подачи и электронную почту подателя. Составное ограничение уникальности на паре «пользователь – курс» исключает повторную подачу заявки.

Алгоритм подачи и рассмотрения заявки. Студент подаёт заявку через соответствующий эндпоинт. Уникальность по паре «пользователь – курс» исключает дублирование. Заявка проходит однонаправленную цепочку статусов: «на рассмотрении» → «одобрена» или «отклонена»; обратные переходы и переходы из конечных статусов запрещены.

При переводе в статус «одобрена» система автоматически создаёт запись о зачислении без привязки к платежу и доставляет студенту уведомление об одобрении. При статусе «отклонена» доступ к курсу не выдаётся.

Специальные курсы не отображаются в публичном каталоге и недоступны для покупки через платёжный цикл.

3.2.8. Подсистема «core» – медиафайлы

Структура данных. Запись «Медиафайл» хранит тип хранилища (объектное хранилище, видеостриминг, внешняя ссылка), идентификатор объекта в хранилище, метаданные (имя файла, тип содержимого, размер), статус (ожидает загрузки, готов, удалён), видимость (приватный, доступен по зачислению, публичный), владельца и счётчик ссылок. Записи «Использование файла» фиксируют связь конкретного файла с конкретной сущностью (задача, урок, курс, пользователь) и роль использования (вложение к задаче, материал урока, обложка курса, аватар пользователя).

Алгоритм жизненного цикла медиафайла. Клиент обращается к эндпоинту инициирования загрузки, указывая тип файла и назначение. Сервер создаёт запись в статусе «ожидает загрузки» и возвращает предподписанную ссылку – временный адрес с встроенной криптографической подписью – для прямой загрузки файла в облачное хранилище без промежуточного прохода через сервер. Переход в статус «готов» происходит через подтверждение клиента или через обработку события из очереди событий хранилища (опрос выполняется фоновой задачей каждые 30 секунд).

Алгоритм периодической очистки. Фоновая задача очистки зависших файлов (запускается каждые 600 секунд) удаляет записи, остающиеся в статусе «ожидает загрузки» дольше 24 часов. Фоновая задача очистки неиспользуемых файлов (запускается каждые 3600 секунд) физически удаляет из хранилища файлы с нулевым счётчиком ссылок, не использовавшиеся более 7 суток.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Защита от преждевременного удаления. Счётчик ссылок увеличивается при добавлении ссылки на файл из любой сущности системы и уменьшается при её удалении. Задача очистки неиспользуемых файлов обрабатывает только файлы с нулевым счётчиком, что предотвращает случайное удаление актуальных данных.

3.3. Описание инфраструктурных компонентов и их параметров

3.3.1. Система управления базами данных PostgreSQL

В качестве реляционного хранилища применяется PostgreSQL версии 15. Все приложения зоны ответственности хранят данные в единой базе; разделение по схемам не применяется, поскольку приложения активно ссылаются друг на друга через внешние ключи. На этапе проектирования созданы ключевые индексы: составной уникальный индекс на паре «студент – домашнее задание» в таблице попыток, индекс на паре «пользователь – курс» в таблице зачислений, индекс по полю срока сдачи в таблице домашних заданий для планировщика напоминаний, а также составной индекс (course_id, created_at) в таблице уведомлений.

3.3.2. Кеш на основе хранилища оперативной памяти Redis

Redis версии 7 используется в трёх ролях с разным временем жизни записей.

1. **Медленный кеш** (время жизни 3600 секунд) – расписание пользователя, публичный каталог курсов.
2. **Стандартный кеш** (время жизни 600 секунд) – содержимое уроков, результаты запросов к каталогу.
3. **Горячий кеш** (время жизни 60 секунд) – часто запрашиваемые данные с быстрым сбросом при изменении.

Дополнительно Redis является хранилищем результатов фоновых задач и каналным слоем для обработчика двусторонних соединений.

3.3.3. Брокер сообщений RabbitMQ

RabbitMQ версии 3 применяется как брокер фоновых задач и шина событий уведомлений. Тематический обменник маршрутизирует события по ключам конкретного пользователя, курса, вебинара и системному каналу. Для изоляции длительных фоновых задач (синхронизация базы знаний, массовые рассылки) используется выделенная очередь, не блокирующая обработку коротких задач.

3.3.4. Сервер обработки запросов Daphne

Серверная часть запускается под управлением асинхронного сервера Daphne за обратным прокси-сервером Nginx. Daphne одновременно обслуживает обычные запросы и постоянные двусторонние соединения (помощник, поток событий). Обработчик фоновых задач запускается с параллелизмом 4; планировщик периодических задач управляет расписанием фоновых операций.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.3.5. Объектное хранилище Yandex Object Storage

Медиафайлы (документы, изображения, файлы к домашним заданиям) хранятся в Yandex Object Storage. Соединение реализовано через стандартные библиотеки для работы с S3-совместимыми хранилищами. Видеоматериалы уроков и записи вебинаров хранятся во внешнем видеохостинге Kinescope с защитой от несанкционированного воспроизведения.

3.4. Описание и обоснование выбора метода организации входных и выходных данных

3.4.1. Входные данные

Запросы к программному интерфейсу. Основным интерфейсом ввода является набор адресов программного интерфейса в формате обмена данными JSON. Все адреса сгруппированы под общим префиксом «/api/» и описаны в формате OpenAPI 3.0 (описание генерируется автоматически и доступно по адресу «/api/schema/»). Аутентификация выполняется по заголовку запроса с токеном доступа. Постоянные соединения (двусторонний канал помощника и поток событий) аутентифицируются по токену в параметре запроса.

Сообщения двустороннего канала. Помощник принимает сообщения в формате JSON с полями «тип команды» и «содержимое». Допустимые типы команд строго проверяются; неизвестный тип возвращает сообщение об ошибке.

В таблице 3.1 приведены ограничения на входные данные в зоне ответственности исполнителя.

Таблица 3.1 – Ограничения на входные данные в зоне ответственности

Входное поле	Ограничение	Способ обработки нарушения
Сообщение помощнику	Не более 1000 символов	Принудительное усечение на стороне клиента
Комментарий преподавателя к задаче	Не более 1500 символов	Принудительное усечение на стороне клиента
Файл к домашнему заданию или уроку	Не более 1 ГБ	Ответ об ошибке
Название файла	Формат «имя.расширение»	Ответ об ошибке
Цена курса	От 0 до 1 000 000 рублей	Принудительное ограничение на стороне клиента
Токен приглашения	64 символа, срок действия не более 3 суток	Ответ об ошибке
Электронная почта в приглашении	Формат «адрес@домен.зона»	Ответ об ошибке

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.4.2. Выходные данные

Ответы программного интерфейса. Стандартный формат ответа – обмен данными JSON. Структура ответа об ошибке унифицирована и содержит поля: признак ошибки, машиночитаемый код ошибки, человекочитаемое сообщение и детали. Внутренние технические детали (текст запросов к базе данных, трассировки программного стека, диагностические сообщения драйвера базы данных) ни при каких условиях не включаются в ответ, возвращаемый клиенту.

Коды состояния ответов: 200/201/204 – для успешных операций; 400 – ошибки проверки входных данных; 401 – требуется вход в систему (токен отсутствует или истёк); 403 – доступ запрещён (недостаточно прав или нет зачисления); 404 – ресурс не найден; 409 – конфликт состояния (повторная попытка, повторная заявка); 429 – превышен лимит запросов; 500 – внутренняя ошибка сервера.

Поток событий. Каждое событие передаётся в стандартном формате: ключевое слово «data», двоеточие, данные в формате JSON, двойной перевод строки. Служебные сигналы для поддержания соединения передаются каждые 30 секунд.

Сообщения двустороннего канала. Ответы помощника: «подключён» (список чатов сессии), «чат создан», «чат удалён», «история получена», «начало генерации», «фрагмент ответа» (с полем «токен»), «генерация завершена», «ошибка».

3.5. Описание и обоснование выбора состава технических и программных средств

3.5.1. Язык программирования Python версии 3.14

Серверная часть написана на языке Python версии 3.14. Выбор обоснован: полной поддержкой со стороны фреймворков Django и Celery; зрелой экосистемой библиотек для всех необходимых интеграций; встроенными средствами асинхронного программирования, необходимыми для обработки двусторонних соединений и потока событий; требованиями учебной программы кафедры.

3.5.2. Веб-фреймворк Django и расширение для программного интерфейса

В качестве основного фреймворка применяется Django версии 6 совместно с расширением для построения программного интерфейса. Выбор обоснован следующими преимуществами.

- **Слой объектно-реляционного отображения с миграциями** сокращает время разработки моделей данных. Все сущности зоны ответственности являются классами слоя отображения и хранятся в таблицах PostgreSQL через автоматически генерируемые миграции.
- **Встроенная система разграничения прав доступа** позволяет реализовать ролевую модель декларативно через классы разрешений на уровне наборов представлений.
- **Механизм сигналов** (сохранения и удаления записи) обеспечивает сброс кеша и планирование напоминаний без жёсткой привязки к месту изменения данных.
- **Расширение автоматической генерации документации** обеспечивает актуальное описание программного интерфейса в формате OpenAPI без ручного сопровождения.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Альтернативные фреймворки рассматривались, но были отвергнуты: FastAPI требует ручной реализации административной панели и механизма сигналов; Flask – ручной интеграции каждого инфраструктурного компонента.

3.5.3. Система фоновых задач Celery и брокер RabbitMQ

Celery версии 5 выбран как стандарт отрасли для проектов на фреймворке Django: полная поддержка периодических задач, маршрутизации по очередям и стратегий повторных попыток. RabbitMQ выбран в качестве брокера сообщений вместо Redis: он обеспечивает семантику «как минимум однократной» доставки с подтверждением получения, что критично для задач с гарантированной доставкой (уведомления, обработка платежей). Тематический обменник поддерживает гибкую маршрутизацию событий уведомлений по нескольким каналам одновременно.

3.5.4. Хранилище оперативной памяти Redis

Redis версии 7 применяется как универсальное кеш-хранилище. Выбор обоснован субмиллисекундным временем отклика, поддержкой атомарных операций и структур данных. Альтернативный Memcached был отвергнут как менее функциональный и не поддерживающий сложные структуры данных.

3.5.5. Система управления базами данных PostgreSQL

PostgreSQL версии 15 выбран за гарантии атомарности, согласованности, изолированности и долговечности транзакций (необходимые при атомарном создании попытки и записи зачисления), поддержку хранения JSON-документов (содержимое урока, метаданные файлов) и активное развитие экосистемы совместимых библиотек.

3.5.6. Языковая модель Yandex GPT и хранилище Yandex Vector Store

Yandex GPT выбрана как доступная российская языковая модель с поддержкой потоковой передачи ответов и программного интерфейса. Yandex Vector Store обеспечивает постоянное хранилище векторных представлений для семантического поиска по материалам курса. Метод дополнения поиском позволяет ограничить ответы помощника тематикой конкретного курса без необходимости дополнительного обучения модели.

3.5.7. Объектное хранилище Yandex Object Storage

Yandex Object Storage выбран за совместимость с протоколом «S3» (использование стандартных библиотек), сеть доставки содержимого на территории Российской Федерации и отсутствие санкционных рисков. MinIO как вариант с самостоятельным хостингом используется в локальной разработке.

3.6. Взаимодействие с другими программами

С серверными приложениями «вебинары» и «статистика» (зона ответственности другого исполнителя) – через прямые ссылки между моделями данных. Запись прогресса прохождения урока

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

(приложение «статистика») ссылается на урок из приложения «курсы»; при сохранении попытки в статусе «отправлена» публикуется сигнал, инициирующий пересчёт прогресса.

С приложением «уведомления» (часть зоны ответственности) – через диспетчер событий, публикующий типизированные события в брокер сообщений. Приложение «уведомления» принимает события и доставляет их конечным пользователям. События вебинарного расписания, публикуемые подсистемой вебинаров (зона ответственности другого исполнителя), обрабатываются тем же диспетчером.

С серверным приложением «корзина» (зона ответственности другого исполнителя) – через прямой программный вызов метода создания платежа. Приложение «корзина» подготавливает список курсов с ценами и передаёт его в подсистему платежей.

С клиентской частью (зона ответственности другого исполнителя) – через программный интерфейс под общим префиксом «/api/», двусторонний канал помощника и поток событий уведомлений.

Схема взаимодействия всех компонентов приведена в Приложении 1 настоящего документа.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Показатели производительности

Ожидаемые показатели производительности серверной части в зоне ответственности определены на основании требований к временным характеристикам индивидуального технического задания и с учётом профиля нагрузки типовой образовательной платформы.

В таблице 4.1 представлены ожидаемые временные характеристики основных групп адресов программного интерфейса.

Таблица 4.1 – Ожидаемые временные характеристики основных групп адресов программного интерфейса

Группа адресов	Медиана (p50)	95-й процентиль (p95)	Подсистема
Публичный каталог курсов (из кеша)	≤ 300 мс	≤ 500 мс	«courses»
Содержимое урока (из кеша)	≤ 300 мс	≤ 500 мс	«courses»
Создание или отправка попытки	≤ 300 мс	≤ 600 мс	«homeworks»
Инициирование оплаты	≤ 300 мс	≤ 600 мс	«payments»
Список уведомлений	≤ 350 мс	≤ 500 мс	«notifications»
Первый фрагмент ответа помощника	≤ 2 с	≤ 3 с	«ai_chat_bot»
Список приглашений или преподавателей	≤ 300 мс	≤ 600 мс	«admin_panel»
Подача или обработка заявки	≤ 300 мс	≤ 600 мс	«applications»
Инициирование загрузки файла (предподписанная ссылка)	≤ 300 мс	≤ 500 мс	«core»
Прочие запросы чтения (списки сущностей)	≤ 750 мс	≤ 1500 мс	Все

Под штатной нагрузкой понимается одновременное обслуживание не более 1000 запросов в секунду и не более 500 активных двусторонних соединений с помощником.

Примечание. Время до первого фрагмента ответа помощника существенно зависит от загрузки внешней языковой модели и сетевой задержки до серверов провайдера. Целевое значение 95-го перцентиля в 3 секунды определено при штатной доступности внешнего программного интерфейса. Длительные операции вынесены в фоновые задачи и не учитываются в показателях синхронных запросов:

- синхронизация содержимого курса в векторное хранилище – не более 5 минут для типового курса (50 уроков);
- доставка уведомления через поток событий от момента публикации события до клиента – не более 3 секунд;
- обработка платежа (включая до 5 повторных попыток) – не более 90 секунд;
- доставка письма по электронной почте – не более 30 секунд (95-й перцентиль);

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- физическое удаление неиспользуемого файла из хранилища – в пределах следующего цикла задачи очистки (не более 3600 секунд).

4.2. Показатели надёжности

Ожидаемые показатели надёжности серверной части в зоне ответственности:

- **Коэффициент готовности** $K_r \geq 0\{, \}995$ (не более 43 часов простоя в год). Достигается архитектурой без состояния на стороне обработчиков запросов, возможностью горизонтального масштабирования и развёртыванием в облачной инфраструктуре.
- **Среднее время между отказами** (MTBF) – не менее 30 суток непрерывной работы при штатной нагрузке.
- **Среднее время восстановления** (MTTR) – не более 15 минут при наличии дежурного оператора.
- **Потеря данных при отказе** приложения – нулевая, поскольку все операции выполняются в атомарных транзакциях базы данных.

Дополнительные средства повышения надёжности:

1. Все фоновые задачи (уведомления, обработка платежей, синхронизация базы знаний, очистка файлов) выполняются с автоматическими повторами при сбое (до 5 попыток с нарастающей задержкой). Временная недоступность любого внешнего сервиса не приводит к необратимой потере операции.
2. Отказ одного из внешних сервисов (Yandex GPT, объектное хранилище, видеохостинг, SMS-сервис, брокер сообщений) не приводит к полной недоступности серверной части. Соответствующие адреса программного интерфейса возвращают понятное сообщение об ошибке.
3. При обрыве потока событий серверная часть корректно освобождает ресурсы брокера сообщений. При повторном подключении клиента доставка возобновляется с учётом актуального состояния очереди.
4. Каждый обработанный запрос записывается в журнал с указанием метода, адреса, кода ответа и времени обработки. Запросы длительностью более 0,5 секунды выделяются в журнале для оперативного анализа.

4.3. Показатели ресурсоёмкости

В таблице 4.2 приведено распределение вычислительных ресурсов по компонентам на рекомендуемой конфигурации сервера развёртывания (виртуальная машина 4 виртуальных процессора / 8 ГБ оперативной памяти).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Таблица 4.2 – Распределение вычислительных ресурсов сервера развёртывания

Компонент	ЦПУ (ядра)	ОЗУ	Назначение
Сервер приложений Daphne (ASGI)	1.5	1.5 ГБ	Обработка HTTP-запросов, двусторонних соединений и потока событий
Обработчик фоновых задач	1.0	1.0 ГБ	Выполнение фоновых задач, синхронизация базы знаний
Планировщик задач	0.1	128 МБ	Расписание периодических фоновых операций
PostgreSQL	1.0	2.0 ГБ	Реляционное хранилище всех сущностей
Redis	0.3	512 МБ	Кеш, каналный слой, хранение результатов задач
RabbitMQ	0.3	512 МБ	Брокер фоновых задач и шина событий уведомлений
Прокси-сервер Nginx	0.2	256 МБ	Приём HTTPS, проксирование к серверу приложений
Итого	4.4	6 ГБ	Резерв 0{,}6 ЦПУ и 2 ГБ ОЗУ для пиковых ситуаций

При выходе нагрузки за пределы штатной предусмотрено горизонтальное масштабирование сервера приложений и обработчика фоновых задач путём запуска дополнительных экземпляров контейнеров.

4.4. Экономические преимущества разработанного решения

Настоящий курсовой проект выполняется в учебных рамках; формальная коммерческая калькуляция не предусмотрена индивидуальным техническим заданием. Тем не менее реализованные технические решения обеспечивают измеримые практические преимущества при коммерческом применении продукта.

4.4.1. Автоматизация проверки домашних заданий

Автоматическая проверка вопросов с вариантами ответа полностью исключает ручную работу преподавателя по всем заданиям тестового типа. По оценкам рынка онлайн-образования, доля тестовых вопросов в типовом курсе составляет от 60 до 80 % от общего числа заданий. Для курса со 100 студентами и 10 домашними заданиями по 5 вопросов каждое экономия составляет около 50 человеко-часов ручной проверки на один курс.

4.4.2. Надёжный платёжный цикл с повторными попытками при сбое

Асинхронная обработка платежей с нарастающими паузами между повторами компенсирует временную недоступность платёжного сервиса без потери транзакции. Статистически это снижает

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

долю «потерянных» платежей из-за технических сбоев с типичных 1–3 % до менее 0{,}1 %, что при 1000 транзакций в месяц составляет от 10 до 30 дополнительно сохранённых конверсий.

4.4.3. Снижение нагрузки на поддержку благодаря контекстному помощнику

Помощник с базой знаний конкретного курса отвечает на типовые вопросы студентов автоматически. По оценкам аналогичных платформ, от 40 до 60 % обращений в службу поддержки посвящены вопросам, ответы на которые присутствуют в учебных материалах. Автоматизация этой доли обращений высвобождает время преподавателя для проверки творческих заданий и индивидуальной работы со студентами.

4.4.4. Сохранение успеваемости через напоминания о сроках сдачи

Автоматические напоминания за 24 часа и за 1 час до срока сдачи домашнего задания повышают долю своевременно сданных работ. Исследования платформ дистанционного образования показывают, что автоматические напоминания увеличивают долю завершивших курс студентов на 15–25 %, что напрямую влияет на удовлетворённость обучающихся и вероятность повторных покупок.

4.5. Ожидаемая потребность

Платформа рассчитана на круглогодичную эксплуатацию без выраженной сезонности. Ожидаемая годовая потребность в рамках учебной инсталляции: не менее 50 курсов в каталоге, не менее 1000 уникальных студентов, не менее 100 преподавателей, не менее 5000 проверенных домашних заданий в год.

При расширении в формате предоставления программного обеспечения как услуги ожидаемая нагрузка составляет до 50 000 студентов и 500 преподавателей на одну инсталляцию. Архитектурные ограничения для данного масштаба устранены на этапе проектирования за счёт отсутствия состояния на стороне обработчиков запросов.

4.6. Сравнение с отечественными и зарубежными аналогами

Полный сравнительный анализ образовательных платформ выполнен в общем командном техническом задании. В настоящем подразделе приведены только функциональные характеристики, непосредственно относящиеся к зоне ответственности исполнителя.

Таблица 4.3 – Сравнение с аналогами по функциональным характеристикам зоны ответственности

Функциональная характеристика	«Про-фес-сия»	«Яндекс Практикум»	Stepik	Coursera	Skill-Box
Иерархия: курс – раздел – урок – домашнее задание	+	+	+	+	+
Автоматическая проверка тестовых заданий	+	+	+	+	+
Ручная проверка задач с развёрнутым ответом	+	+	+	+	+

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Функциональная характеристика	«Про- фес-сия»	«Яндекс Прак- тикум»	Stepik	Coursera	Skill-Box
Помощник с контекстом конкретного курса	+	+	–	–	–
Уведомления в реальном времени через поток событий	+	?	–	–	–
Напоминания о сроках сдачи через почту и SMS	+	+	+	+	+
Заявки на специальные курсы без оплаты	+	–	+	–	–
Предподписанные ссылки для загрузки файлов	+	?	?	+	?
Периодическая очистка неиспользуемых файлов	+	?	?	?	?

Примечание. Знак «+» – характеристика присутствует; «–» – отсутствует; «?» – сведения публично не подтверждены.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

5.1. Стандарты Единой системы программной документации

1. ГОСТ 19.101–77 Единая система программной документации. Виды программ и программных документов. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.103–77 Единая система программной документации. Обозначения программ и программных документов. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.105–78 Единая система программной документации. Общие требования к программным документам. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.106–78 Единая система программной документации. Требования к программным документам, выполненным печатным способом. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.201–78 Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – М.: Стандартинформ, 2010.
6. ГОСТ 19.301–79 Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению. – М.: Стандартинформ, 2010.
7. ГОСТ 19.401–78 Единая система программной документации. Текст программы. Требования к содержанию и оформлению. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.404–79 Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. – М.: Стандартинформ, 2010.
9. ГОСТ 19.504–79 Единая система программной документации. Руководство программиста. Требования к содержанию и оформлению. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.603–78 Единая система программной документации. Общие правила внесения изменений. – М.: ИПК Издательство стандартов, 2001.

5.2. Документация веб-фреймворка и инфраструктуры серверной части

1. Django Documentation, version 6. Официальная документация веб-фреймворка Django. URL: <https://docs.djangoproject.com/en/6.0/> (дата обращения: 10.05.2026).
2. Django REST Framework Documentation. Официальная документация фреймворка REST-интерфейсов на основе Django. URL: <https://www.django-rest-framework.org/> (дата обращения: 10.05.2026).
3. Django Channels Documentation. Расширение Django для WebSocket и SSE. URL: <https://channels.readthedocs.io/> (дата обращения: 10.05.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. drf-spectacular Documentation. Автоматическая генерация OpenAPI-схемы для Django REST Framework. URL: <https://drf-spectacular.readthedocs.io/> (дата обращения: 10.05.2026).
5. django-rest-framework-simplejwt Documentation. JWT-аутентификация для Django REST Framework. URL: <https://django-rest-framework-simplejwt.readthedocs.io/> (дата обращения: 10.05.2026).
6. Celery Documentation, version 5. Официальная документация системы асинхронных задач. URL: <https://docs.celeryq.dev/en/stable/> (дата обращения: 10.05.2026).
7. Daphne Documentation. ASGI-сервер для Django Channels. URL: <https://github.com/django/daphne> (дата обращения: 10.05.2026).
8. PostgreSQL 15 Documentation. Официальная документация СУБД PostgreSQL. URL: <https://www.postgresql.org/docs/15/> (дата обращения: 10.05.2026).
9. Redis 7 Documentation. Официальная документация Redis. URL: <https://redis.io/docs/latest/> (дата обращения: 10.05.2026).
10. RabbitMQ 3 Documentation. Официальная документация брокера сообщений RabbitMQ. URL: <https://www.rabbitmq.com/documentation.html> (дата обращения: 10.05.2026).
11. Docker Documentation. Compose file reference. URL: <https://docs.docker.com/compose/> (дата обращения: 10.05.2026).

5.3. Документация внешних сервисов

1. Yandex Foundation Models Documentation. Документация языковых моделей Яндекс и векторного хранилища. URL: <https://cloud.yandex.ru/docs/foundation-models/> (дата обращения: 10.05.2026).
2. Yandex Object Storage Documentation. Документация S3-совместимого объектного хранилища Яндекс. URL: <https://yandex.cloud/ru/docs/storage/> (дата обращения: 10.05.2026).
3. Kinescope Documentation. Документация видеохостинга с защитой воспроизведения. URL: <https://docs.kinescope.io/> (дата обращения: 10.05.2026).
4. Notificore Documentation. Документация сервиса доставки SMS. URL: <https://notificore.ru/> (дата обращения: 10.05.2026).

5.4. Стандарты и спецификации

1. Jones M., Bradley J., Sakimura N. RFC 7519: JSON Web Token (JWT). IETF, May 2015. URL: <https://tools.ietf.org/html/rfc7519> (дата обращения: 10.05.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. Hardt D. RFC 6749: The OAuth 2.0 Authorization Framework. IETF, October 2012. URL: <https://tools.ietf.org/html/rfc6749> (дата обращения: 10.05.2026).
3. OpenAPI Specification, version 3.0.3. URL: <https://spec.openapis.org/oas/v3.0.3> (дата обращения: 10.05.2026).
4. Fielding R. T., Reschke J. RFC 7230–7235: HTTP/1.1. IETF, June 2014.

5.5. Прочие источники

1. Python 3.14 Documentation. URL: <https://docs.python.org/3.14/> (дата обращения: 10.05.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1

ОПИСАНИЕ ВНЕШНИХ ВЗАИМОДЕЙСТВИЙ КОМПОНЕНТОВ

В настоящем приложении описаны все внешние взаимодействия серверной части 1 с другими компонентами платформы «Профессия» и внешними сервисами. Для каждого взаимодействия указаны используемый протокол, направление и назначение.

Взаимодействие с клиентской частью

Клиентская часть платформы (браузерное приложение на React) взаимодействует с серверной частью 1 по трём независимым протоколам, каждый из которых решает свою задачу.

Обычные HTTP-запросы используются для всех операций чтения и записи данных: получение каталога курсов, отправка попыток домашних заданий, создание платежа, управление заявками и приглашениями. Клиент отправляет запрос, сервер немедленно возвращает ответ в формате JSON и закрывает соединение. Все такие адреса сгруппированы под префиксом «/api/» и задокументированы в формате OpenAPI 3.0 по адресу «/api/schema/». Аутентификация выполняется через заголовок запроса с JWT-токеном доступа. Токен действует 1 час; при истечении клиент обновляет его через Refresh Token (срок действия – 7 суток) с автоматической ротацией.

Постоянное двустороннее соединение (WebSocket) используется исключительно для канала ИИ-помощника. Соединение устанавливается по адресу «/ws/chat/{course_slug}/» при переходе пользователя в раздел помощника. В отличие от обычных запросов, это соединение остаётся открытым на протяжении всей сессии работы с помощником: клиент может отправлять команды в любой момент, а сервер отправляет ответы частями по мере их генерации языковой моделью. Аутентификация выполняется через JWT-токен, переданный в параметре строки подключения (так как браузерный WebSocket API не поддерживает произвольные заголовки). При закрытии вкладки браузера соединение корректно завершается на стороне сервера. Поддерживаемые команды от клиента: создать новый чат, удалить чат, получить историю сообщений, отправить сообщение. Каждый фрагмент ответа языковой модели немедленно пересылается клиенту отдельным сообщением, не дожидаясь завершения генерации.

Поток событий сервера (SSE) используется для доставки уведомлений в режиме реального времени. Клиент открывает соединение по адресу «/api/notifications/stream/» и держит его открытым. Сервер самостоятельно отправляет сообщения при возникновении событий: новое домашнее задание, изменение срока сдачи, результат проверки попытки, системные уведомления. В отличие от WebSocket, поток событий однонаправленный: клиент только слушает, не отправляя ничего обратно. Каждые 30 секунд сервер отправляет служебный сигнал для предотвращения разрыва соединения промежуточными прокси-серверами. Аутентификация выполняется через JWT-токен в параметре запроса. При обрыве соединения браузер автоматически восстанавливает подключение по стандартному поведению протокола.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Взаимодействие с инфраструктурными компонентами

PostgreSQL 15 является единственным постоянным хранилищем всех структурированных данных. Соединение устанавливается через драйвер «psycopg2» по протоколу TCP. Все операции с базой данных выполняются через слой объектно-реляционного отображения Django ORM, который транслирует вызовы Python-кода в SQL-запросы и обеспечивает атомарность транзакций.

Redis 7 используется в трёх ролях одновременно. Во-первых, как кеш для хранения результатов частых запросов (каталог курсов, содержимое уроков, расписание) с разным временем жизни: 3600 секунд для редко меняющихся данных, 600 секунд для большинства запросов, 60 секунд для горячих данных. Во-вторых, как канальный слой для Django Channels: Redis хранит связь между конкретным WebSocket-соединением и процессом Daphne, который его обслуживает, что позволяет направить ответ в нужное соединение при горизонтальном масштабировании. В-третьих, как хранилище результатов выполнения Celery-задач.

RabbitMQ 3 используется как брокер очереди фоновых задач Celery и как шина событий уведомлений. Для задач Celery RabbitMQ гарантирует, что каждая задача будет выполнена хотя бы один раз: задача удаляется из очереди только после получения подтверждения о завершении. Для уведомлений создан тематический обменник с маршрутизацией по ключам: «user.{id}» для личных уведомлений, «course.{id}» для курсовых, «system.all» для системных, «webinar.{id}» для вебинарных событий. Эндпоинт потока событий подписывается сразу на все нужные ключи и публикует полученные сообщения в открытое SSE-соединение.

Взаимодействие с внешними сервисами

В таблице П1.1 приведено описание каждого внешнего взаимодействия серверной части 1.

Таблица П1.1 – Описание внешних взаимодействий серверной части 1

Внешний компонент	Протокол	Назначение
Клиентская часть (браузер)	HTTP + JSON	Основной интерфейс взаимодействия со всеми подсистемами
Клиентская часть (браузер)	WebSocket	Двусторонний канал помощника на основе языковой модели
Клиентская часть (браузер)	SSE	Поток событий-уведомлений в режиме реального времени
PostgreSQL 15	TCP/psycopg2	Хранение всех сущностей зоны ответственности
Redis 7	TCP/redis-py	Кеш, канальный слой, хранение результатов задач
RabbitMQ 3	AMQP	Брокер фоновых задач и шина событий уведомлений
Yandex GPT	HTTPS	Генерация потоковых ответов помощника
Yandex Vector Store	HTTPS	База знаний курса для метода дополнения поиском

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Внешний компонент	Протокол	Назначение
Yandex Object Storage	HTTPS/S3	Хранение медиафайлов; предподписанные ссылки для загрузки
Kinescope	HTTPS	Загрузка видеоматериалов, авторизация воспроизведения
«Notifcore»	HTTPS	Доставка SMS-уведомлений (сроки сдачи, результаты)
Почтовый сервер (SMTP)	SMTP	Доставка уведомлений по электронной почте

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2

ПЕРСОНАЛЬНЫЙ ВКЛАД И ГРАНИЦЫ ОТВЕТСТВЕННОСТИ

В настоящем приложении фиксируется персональный вклад исполнителя С.М. Павлычева в разработку серверной части 1 платформы «Профессия».

Таблица П2.1 – Матрица ответственности исполнителя С.М. Павлычева

Область работы	Конкретное содержание	Результат
Каталог и содержимое курсов	Модели данных: «Курс», «Раздел», «Урок», «Домашнее задание», «Вопрос», «Задача», «Зачисление»; сигналы сброса кеша; кеширование каталога; разграничение прав доступа на уровне создания, чтения, обновления и удаления	Приложение «courses» с полным набором адресов программного интерфейса
Домашние задания и попытки	Модели данных: «Попытка», «Ответ на вопрос», «Ответ на задачу», «Оценка преподавателя»; сервисы: атомарное создание попытки, автоматическая проверка, ручная проверка	Приложение «homeworks» с автоматической и ручной проверкой
Платежи	Модели данных: «Платёж», «Позиция платежа», «Зачисление»; подменный платёжный сервис; фоновая задача обработки платежа с повторами	Приложение «payments» с асинхронной обработкой
Уведомления	Модель «Уведомление»; диспетчер событий; поток событий через брокер сообщений; отправка по почте; отправка по SMS	Приложение «notifications» с многоканальной доставкой
Помощник на основе языковой модели	Обработчик двусторонних соединений; сервисы: обращение к языковой модели, синхронизация базы знаний; потоковая генерация ответов	Приложение «ai_chat_bot» со стримингом Yandex GPT
Панель модератора	Модель «Приглашение» с токенами и сроком действия; управление составом авторов курсов; публикация и снятие с публикации	Приложение «admin_panel»
Заявки на специальные курсы	Модель «Заявка»; конечный автомат статусов; автоматическое зачисление при одобрении	Приложение «applications»
Медиафайлы	Модели «Медиафайл» и «Использование файла»; предподписанные ссылки; три фоновые задачи очистки	Приложение «core»

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Ключевые проектные решения исполнителя

1. **Единый пулл уведомлений.** Все события платформы, требующие уведомления пользователя, проходят через единый диспетчер. Это исключает дублирование логики уведомлений в разных приложениях и позволяет добавлять новые каналы доставки (например, мессенджеры) без изменения бизнес-логики.
2. **Атомарное создание попытки.** Операция «получить существующую или создать новую» на уровне базы данных с ограничением уникальности гарантирует, что даже при одновременных параллельных запросах студент не получит две попытки на одно задание.
3. **Подменный платёжный шлюз через единый интерфейс.** Весь код обработки платежей работает с абстрактным интерфейсом сервиса, а не напрямую с конкретным провайдером. Это позволяет заменить учебный подменный сервис на реальный платёжный шлюз одним изменением настройки.
4. **Периодическая очистка медиафайлов с отсрочкой.** Двухступенчатая очистка (сначала зависшие файлы через 24 часа, затем неиспользуемые через 7 суток) предотвращает накопление «мусора» в объектном хранилище без риска случайно удалить файл, который ещё используется.
5. **Управление параллелизмом при обращении к языковой модели.** Механизм семафора ограничивает число одновременных обращений к Yandex GPT, что предотвращает превышение установленных провайдером квот при пиковой нагрузке на платформу.
6. **Трёхуровневый кеш с разным временем жизни.** Разделение кеша на «холодный» (для редко меняющихся данных), «стандартный» (для большинства запросов) и «горячий» (для часто меняющихся данных) позволяет гибко балансировать между актуальностью данных и нагрузкой на базу данных.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3 ГЛОССАРИЙ И ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

В настоящем приложении приведены термины, сокращения и аббревиатуры, используемые в настоящем документе. Термины расположены в порядке первого упоминания в тексте и сгруппированы по тематике.

Общие понятия и роли

Платформа – Настоящее программное изделие «Веб-приложение для онлайн-обучения с поддержкой видеосвязи и ИИ-помощника», краткое наименование – «Профессия».

Студент – Пользователь платформы с ролью «ученик», проходящий курсы и выполняющий домашние задания.

Преподаватель – Пользователь платформы с ролью «преподаватель», создающий учебные курсы и проверяющий домашние задания студентов.

Модератор – Пользователь платформы с расширенными административными правами: управляет составом преподавателей, публикует и снимает курсы с публикации, одобряет или отклоняет заявки на любой курс.

Зачисление – Запись в базе данных, подтверждающая право студента на доступ к материалам конкретного курса. Создаётся после успешной оплаты или одобрения заявки. Имеет срок действия и источник (оплата или заявка).

Попытка – Запись о попытке студента выполнить домашнее задание. Проходит три состояния: «черновик» – студент вводит ответы; «отправлена» – студент завершил выполнение; «проверена» – преподаватель выставил итоговую оценку.

ДЗ – Домашнее задание – учебное задание, прикрепленное к уроку. Может содержать вопросы с вариантами ответа (проверяются автоматически) и задачи с развёрнутым ответом (проверяются преподавателем вручную).

Заявка – Запрос студента на зачисление в специальный курс без оплаты. Проходит однонаправленную цепочку статусов: «на рассмотрении» → «одобрена» или «отклонена».

Повторная подача заявки на тот же курс не допускается.

Срок сдачи (дедлайн) – Дата и время, до наступления которых студент обязан отправить выполненное домашнее задание. При изменении срока система автоматически обновляет запланированные напоминания.

Программный интерфейс и протоколы взаимодействия

API (Программный интерфейс) – Application Programming Interface – набор правил и адресов, по которым программы обмениваются данными. В настоящем документе – набор адресов серверной части, по которым клиентское приложение обращается к серверу.

REST – Representational State Transfer – стиль построения программных интерфейсов поверх протокола HTTP. Клиент обращается к ресурсам (курсам, попыткам, платежам) по стандартным командам: получить, создать, обновить, удалить.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Эндпоинт – Конкретный адрес на сервере (URL), по которому клиентское приложение выполняет определённую операцию. Например, адрес «/api/courses/» используется для получения списка курсов.

HTTP – HyperText Transfer Protocol – протокол передачи данных между клиентом и сервером во Всемирной паутине. Является основой взаимодействия браузера с серверной частью.

Код состояния HTTP – Трёхзначное число в ответе сервера, обозначающее результат выполнения запроса. Основные коды: 200 – успешно выполнено; 201 – создан новый объект; 204 – успешно, без данных в ответе; 400 – ошибка в данных запроса; 401 – необходимо войти в систему; 403 – доступ запрещён; 404 – ресурс не найден; 409 – конфликт (например, попытка создать дубликат); 429 – слишком много запросов; 500 – внутренняя ошибка сервера.

JSON – JavaScript Object Notation – текстовый формат представления структурированных данных. Используется как основной формат обмена данными между клиентской и серверной частями платформы.

OpenAPI – Стандарт машиночитаемого описания программных интерфейсов. Позволяет автоматически генерировать документацию, тесты и клиентский код по описанию эндпоинтов.

WebSocket (Двустороннее соединение) – Протокол постоянного двустороннего соединения между клиентом и сервером поверх TCP. В отличие от обычных HTTP-запросов, позволяет серверу самостоятельно отправлять данные клиенту без ожидания запроса. Используется для канала помощника на основе языковой модели.

SSE (Поток событий сервера) – Server-Sent Events – механизм постоянного одностороннего соединения, при котором сервер самостоятельно отправляет клиенту обновления по открытому соединению. Используется для доставки уведомлений в режиме реального времени без необходимости периодических запросов со стороны клиента.

Webhook – Автоматическое уведомление от внешнего сервиса (например, платёжной системы) на заранее известный адрес сервера о наступлении некоторого события. Сервер принимает webhook и обрабатывает его как обычный входящий запрос.

JWT (Токен аутентификации) – JSON Web Token – цифровой жетон, подтверждающий личность вошедшего пользователя. Содержит зашифрованную информацию о пользователе и подпись сервера. Передаётся в заголовке каждого запроса вместо повторного ввода пароля. Имеет ограниченное время действия.

HTTPS – HTTP Secure – защищённая версия протокола HTTP с шифрованием трафика.

Обеспечивает конфиденциальность и целостность передаваемых данных.

AMQP – Advanced Message Queuing Protocol – протокол передачи сообщений через брокер.

Используется для взаимодействия серверной части с брокером сообщений RabbitMQ.

SMTP – Simple Mail Transfer Protocol – протокол передачи электронной почты. Используется серверной частью для отправки уведомлений по электронной почте через почтовый сервер.

S3 – Simple Storage Service – протокол и интерфейс для работы с объектным хранилищем файлов, разработанный компанией Amazon. Yandex Object Storage реализует совместимый с этим протоколом интерфейс, что позволяет использовать стандартные клиентские библиотеки.

Предподписанная ссылка – Временный адрес с встроенной криптографической подписью, позволяющий загрузить файл напрямую в облачное хранилище без промежуточного прохода через сервер платформы. Действует ограниченное время.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Архитектурные паттерны и технологии

RBAC (Ролевое разграничение доступа) – Role-Based Access Control – способ управления доступом, при котором права пользователя определяются его ролью в системе. В платформе: «студент» может читать материалы; «преподаватель» может создавать курсы и проверять работы; «модератор» может управлять преподавателями и публикацией курсов.

ORM (Слой объектно-реляционного отображения) – Object-Relational Mapping – технология, позволяющая работать с данными базы данных через объекты языка программирования, не составляя SQL-запросы вручную. Фреймворк Django предоставляет встроенный слой объектно-реляционного отображения.

RAG (Метод дополнения поиском) – Retrieval-Augmented Generation – метод работы с языковой моделью, при котором перед формированием ответа система выполняет поиск релевантных фрагментов в базе знаний (материалах курса) и передаёт их модели как дополнительный контекст. Это позволяет ограничить ответы тематикой курса и повысить их точность без дополнительного обучения модели.

Векторное представление (эмбединг) – Числовой вектор высокой размерности, кодирующий смысловое содержание текстового фрагмента таким образом, что семантически близкие тексты имеют близкие векторы. Используется для семантического поиска в базе знаний курса.

Идемпотентность – Свойство операции, при котором повторное её выполнение даёт тот же результат, что и первое. Например, повторный запрос на создание черновика попытки возвращает уже существующий черновик, не создавая дубликат.

Атомарная операция – Операция, выполняемая как единое неделимое действие: она либо выполняется полностью, либо не выполняется вовсе. В базах данных обеспечивается механизмом транзакций.

Сигнал (в контексте фреймворка Django) – Механизм оповещения: при наступлении определённого события (сохранение или удаление записи) фреймворк автоматически вызывает зарегистрированные обработчики. Используется для сброса кеша и планирования напоминаний.

Миграция базы данных – Набор инструкций по изменению структуры базы данных (добавление таблиц, столбцов, индексов), сгенерированных автоматически на основе изменений в описаниях моделей данных и применяемых последовательно.

Конечный автомат статусов – Модель, описывающая допустимые состояния объекта и разрешённые переходы между ними. Например, заявка может перейти из «на рассмотрении» в «одобрена» или «отклонена», но не обратно.

Мягкое удаление – Способ «удаления» объекта без физического удаления записи из базы данных: запись помечается специальным признаком и исключается из всех запросов. Позволяет сохранить историческую целостность связей.

Инфраструктурные компоненты

Кеш (кеширование) – Временное хранение результатов запросов для ускорения повторного обращения. При наличии актуального кеша сервер возвращает сохранённый результат, не обращаясь к базе данных. Время жизни записи в кеше определяется параметром TTL.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

TTL (Время жизни) – Time To Live – промежуток времени, по истечении которого запись в кеше считается устаревшей и удаляется. После истечения TTL следующий запрос вновь обращается к базе данных.

Брокер сообщений – Программа-посредник, принимающая сообщения от одних компонентов системы и доставляющая их другим по заданным правилам маршрутизации. В платформе используется RabbitMQ для очереди фоновых задач и шины событий уведомлений.

Очередь фоновых задач – Механизм асинхронного выполнения операций, которые не требуют немедленного ответа пользователю: отправка уведомлений, обработка платежей, синхронизация базы знаний. Задача помещается в очередь и выполняется отдельным процессом-обработчиком.

Экспоненциальная задержка при повторях – Стратегия повторного выполнения задачи при сбое, при которой время ожидания перед каждой следующей попыткой увеличивается: 15 с, 30 с, 60 с и так далее. Снижает нагрузку на временно недоступный сервис.

ASGI – Asynchronous Server Gateway Interface – стандарт асинхронного интерфейса между Python-приложением и веб-сервером. В отличие от синхронного WSGI, поддерживает длительные соединения (WebSocket, SSE). Реализован сервером Daphne.

Контейнеризация (Докер) – Технология изолированного запуска программных компонентов в контейнерах – лёгких виртуальных средах с предустановленными зависимостями. Docker Compose позволяет описать и запустить несколько контейнеров (база данных, кеш, брокер, сервер приложений) единой командой.

Прокси-сервер – Промежуточный сервер, принимающий входящие запросы и перенаправляющий их к серверу приложений. Nginx в составе платформы принимает HTTPS-запросы, отдаёт статические файлы и проксирует динамические запросы к Daphne.

Горизонтальное масштабирование – Увеличение производительности системы путём добавления дополнительных экземпляров сервера приложений или обработчика задач, а не замены существующего более мощным.

DRM (Защита воспроизведения) – Digital Rights Management – технология защиты цифрового контента от несанкционированного воспроизведения и копирования. Видеоматериалы уроков хранятся в видеохостинге Kinescope с DRM-защитой.

Сокращения

ЕСПД – Единая система программной документации – комплекс государственных стандартов России, регулирующих состав и оформление программной документации.

ГОСТ – Государственный стандарт.

ПЗ – Пояснительная записка.

ТЗ – Техническое задание.

ПМИ – Программа и методика испытаний.

СУБД – Система управления базами данных.

ЦПУ – Центральный процессор (CPU – Central Processing Unit).

ОЗУ – Оперативное запоминающее устройство (RAM – Random Access Memory).

ГБ – Гигабайт – единица измерения объёма информации, равная 10^9 байт.

SMS – Short Message Service – служба коротких текстовых сообщений.

p50 (медиана) – Значение метрики, которое не превышает 50 % измерений. Означает, что половина запросов обрабатывается быстрее указанного значения.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

p95 (95-й процентиль) – Значение метрики, которое не превышает 95 % измерений. Означает, что подавляющее большинство запросов (95 из 100) обрабатывается не медленнее указанного значения.

MTBF – Mean Time Between Failures – среднее время между отказами; показатель надёжности системы.

MTTR – Mean Time To Recovery – среднее время восстановления после отказа.

TLS – Transport Layer Security – протокол шифрования транспортного уровня, обеспечивающий безопасность соединения HTTPS.

URL – Uniform Resource Locator – унифицированный указатель ресурса; адрес в сети.

CRUD – Create, Read, Update, Delete – четыре базовые операции с данными: создание, чтение, обновление, удаление.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ОПИСАНИЕ БАЗЫ ДАННЫХ

В настоящем приложении база данных PostgreSQL 15, схема «public», именование таблиц по правилу Django «{приложение}_{модель}». Состав полей наглядно представлен на рисунках П4.1–П4.7; под каждым рисунком приведены Кардинальность связей и прикладная (предметная) логика по исходному коду серверной части (backend/apps/). Полный перечень таблиц зоны ответственности – в таблице П4.8.

Фрагменты ER-диаграммы, Кардинальность связей и бизнес-логика

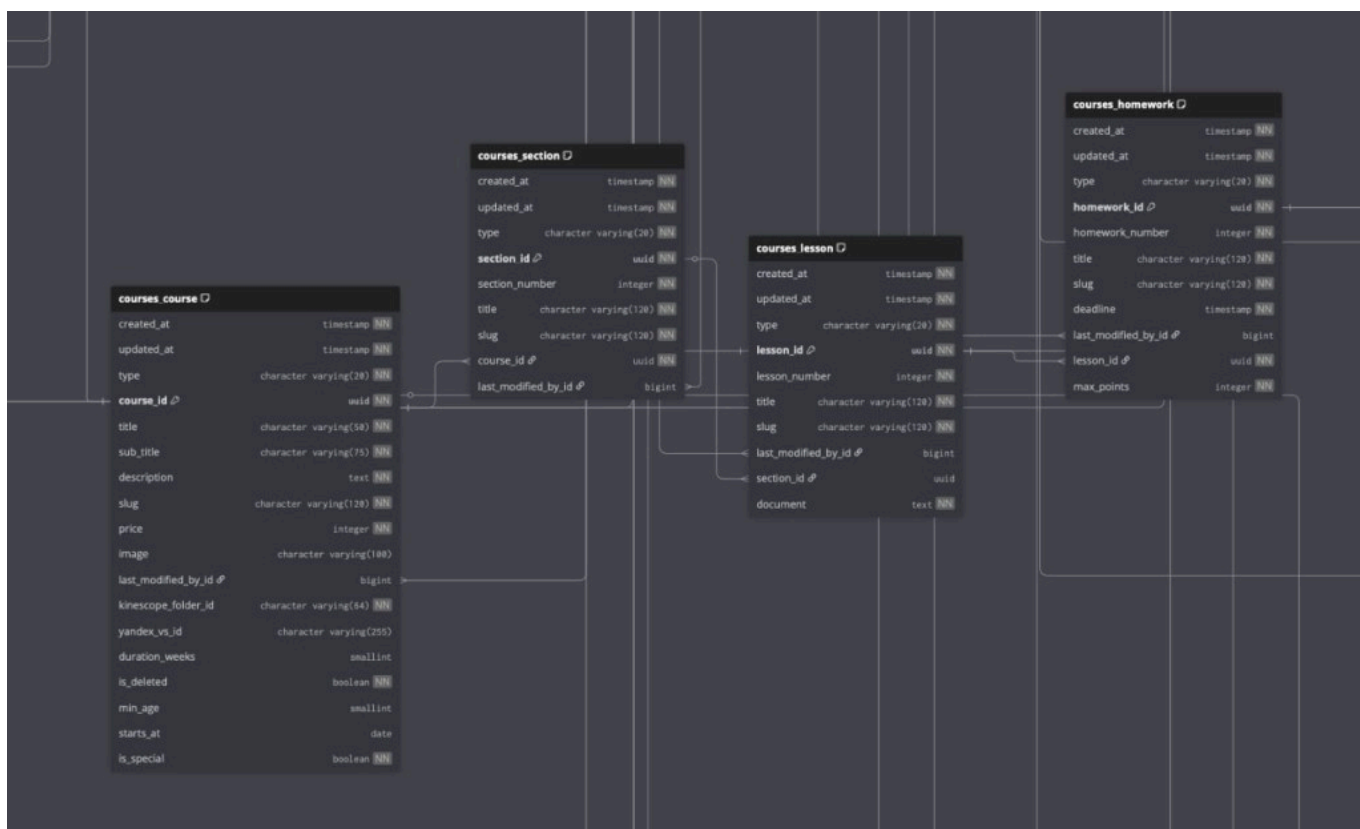


Рисунок П4.1 – Фрагмент ER-диаграммы: иерархия «курс – раздел – урок – домашнее задание»

Рисунок П4.1 – Кардинальность связей и логика приложения

1. **Кардинальность:** `courses_course` → `courses_section` и далее по цепочке до `courses_homework` – связи 1:N по внешним ключам родителя; поле `last_modified_by_id` даёт N:1 на `users_user` для сущностей с аудитом последнего изменения.
2. **Логика:** уровни «курс – секция – урок – домашка» отражают продуктовую иерархию материалов; порядковые номера секций и уроков выставляются при сохранении

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

моделей (AutoIncrementMixin в `courses/models.py`). Курс различает черновик и публикацию, поддерживает мягкое исключение (`is_deleted`), спецкурсы (`is_special`). Содержимое урока хранится как JSON в `document`; при выдаче клиенту вложения резолвятся в URL подсистемой доступа к медиа.

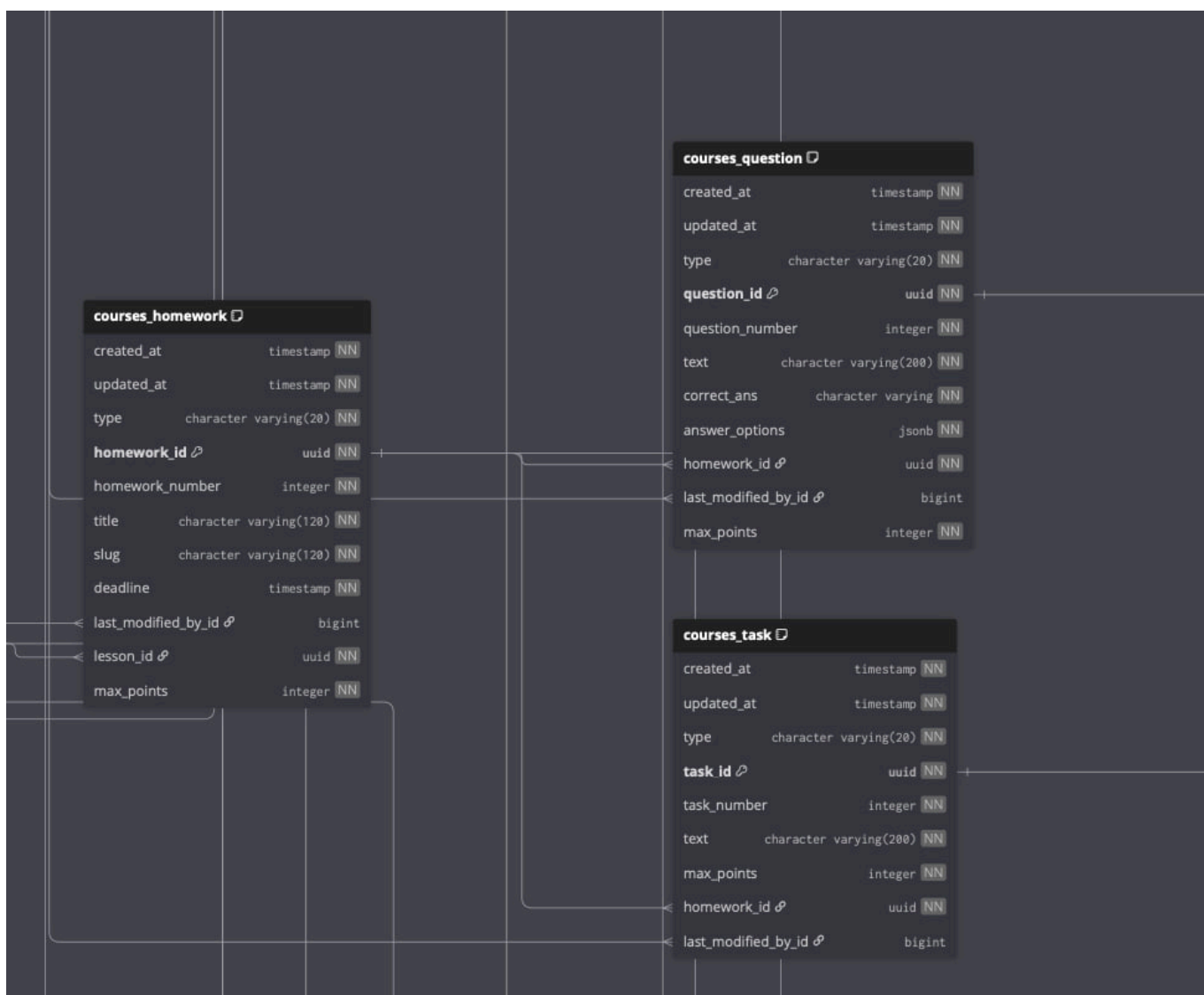


Рисунок П4.2 – Фрагмент ER-диаграммы: состав домашнего задания (вопросы и задачи)

Рисунок П4.2 – Кардинальность связей и логика приложения

- Кардинальность:** `courses_homework` → `courses_question` и `courses_homework` → `courses_task` – по два отношения 1:N (два набора потомков у одной домашней работы).
- Логика:** вопросы с фиксированным правильным ответом и вариантами (`answer_options`, `correct_ans`) проверяются автоматически при сдаче; задачи (`courses_task`) проверяются преподавателем. Сумма `max_points` по вопросам и

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

задачам согласуется с полем max_points домашнего задания (recalc_max_points в модели Homework).



Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Рисунок П4.3 – Фрагмент ER-диаграммы: пользователь, курс, зачисления, заявки, авторы

Рисунок П4.3 – Кардинальность связей и логика приложения

1. **Кардинальность:** `users_user` ↔ `courses_course` через `courses_course_authors` – M:N; `users_user` → `course_enrollments`, `courses_course` → `course_enrollments` – 1:N; опционально `payments` → `course_enrollments` (поле `payment_id`); те же субъекты связаны с `course_applications` отношениями 1:N (податель, рассмотревший, курс).
2. **Логика:** одна пара (студент, курс) не может иметь двух записей зачисления (`unique_together` в `CourseEnrollment`); источник доступа кодируется `source` (`payment` / `application`), при зачислении по заявке внешний ключ на платёж пустой. Заявка на спецкурс проходит статусы без обратных переходов; при одобрении создаётся зачисление без оплаты. Авторы курса задаются связью M:N с пользователями-учителями.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

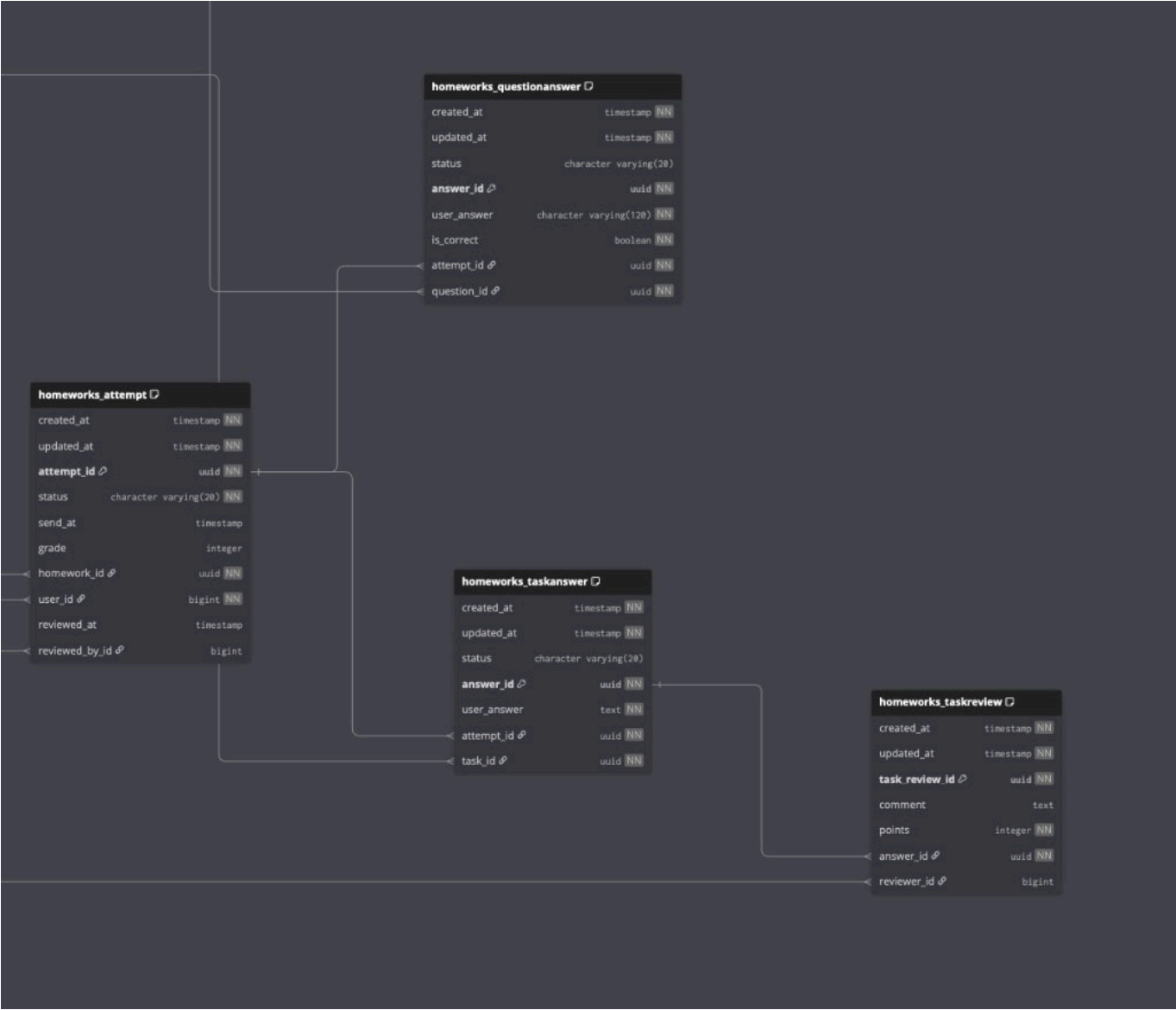


Рисунок П4.4 – Фрагмент ER-диаграммы: попытка сдачи домашнего задания, ответы и рецензии

Рисунок П4.4 – Кардинальность связей и логика приложения

1. **Кардинальность:** homeworks_attempt связана с заданием и студентом N:1; уникальность (user_id, homework_id) сокращает множество попыток в БД до одной записи на пару; homeworks_attempt → homeworks_questionanswer и → homeworks_taskanswer – 1:N; homeworks_taskanswer → homeworks_taskreview – 1:1 (уникальность answer_id у рецензии в DDL).
2. **Логика:** черновик попытки создаётся идемпотентно (get_or_create по студенту и заданию); отправка переводит статус, запускает автопроверку вопросов и блокирует сдачу при неготовых медиа. Рецензия на задачу одна на ответ; после проверки всех задач пересчитывается итог и рассылается событие уведомления.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

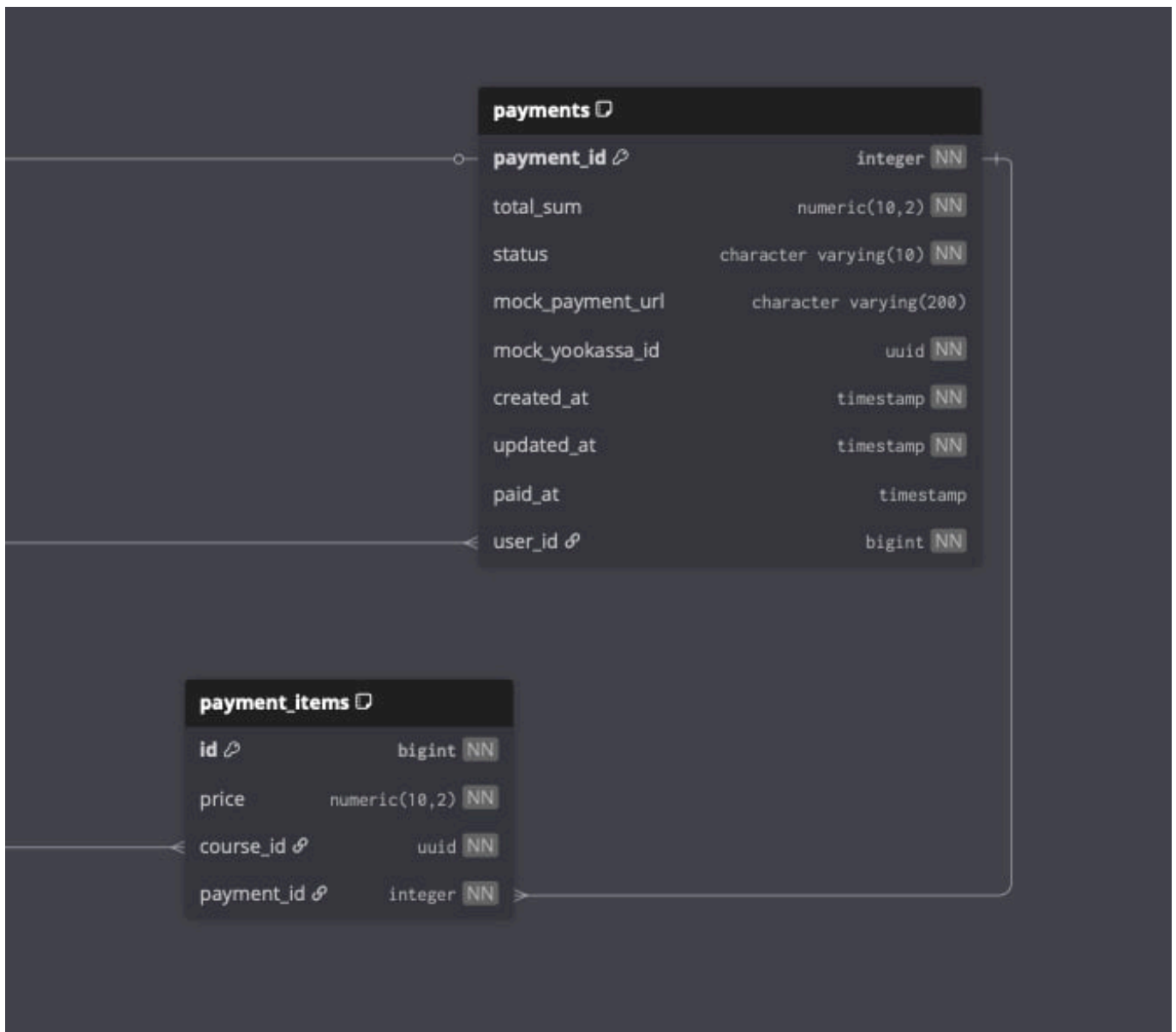


Рисунок П4.5 – Фрагмент ER-диаграммы: платёж и позиции платежа

Рисунок П4.5 – Кардинальность связей и логика приложения

1. **Кардинальность:** `users_user` → `payments` 1:N; `payments` → `payment_items` 1:N; `courses_course` → `payment_items` 1:N; уникальность пары (`payment_id`, `course_id`) не допускает дубли курса в одном платеже.
2. **Логика:** сущность платежа создаётся при оплате корзины; позиции фиксируют цену на момент покупки. Статус и идентификатор подменного шлюза обрабатываются асинхронно (Celery); при успехе создаются зачисления с `source = payment` и политикой срока доступа курса.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

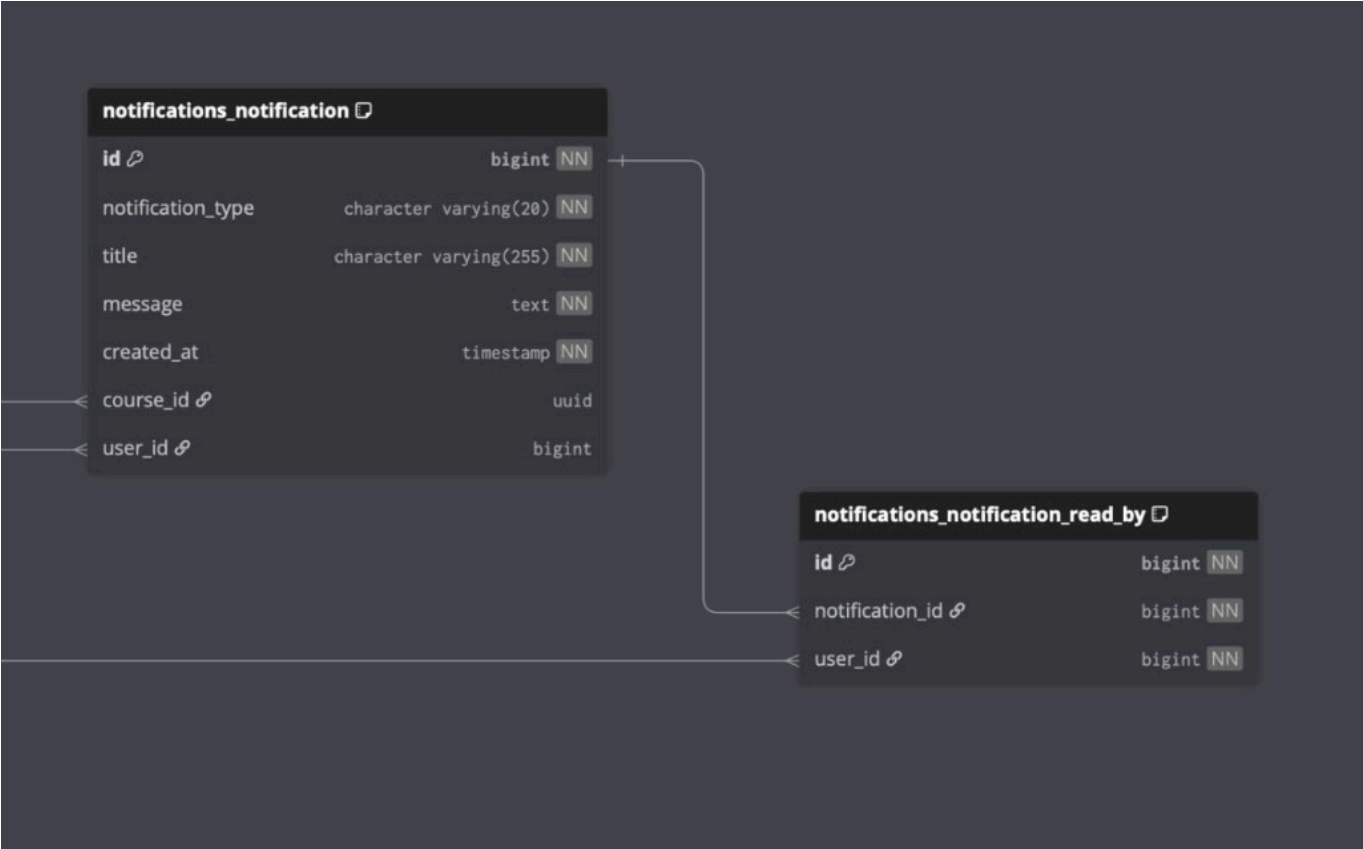


Рисунок П4.6 – Фрагмент ER-диаграммы: уведомление и факт прочтения

Рисунок П4.6 – Кардинальность связей и логика приложения

- 1. **Кардинальность:** уведомление 1:N к строкам notifications_notification_read_by; пользователь 1:N к тем же строкам; опциональные связи уведомления с пользователем и курсом (персональные, курсовые, системные типы).
- 2. **Логика:** фиксируется текст и тип события; факт прочтения конкретным пользователем хранится отдельной записью (реализация пересечения «кто прочитал»). Те же события дублируются в другие каналы (SSE, почта, SMS) через общий диспетчер, оркестрация – в коде приложения notifications, а не только в этих таблицах.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

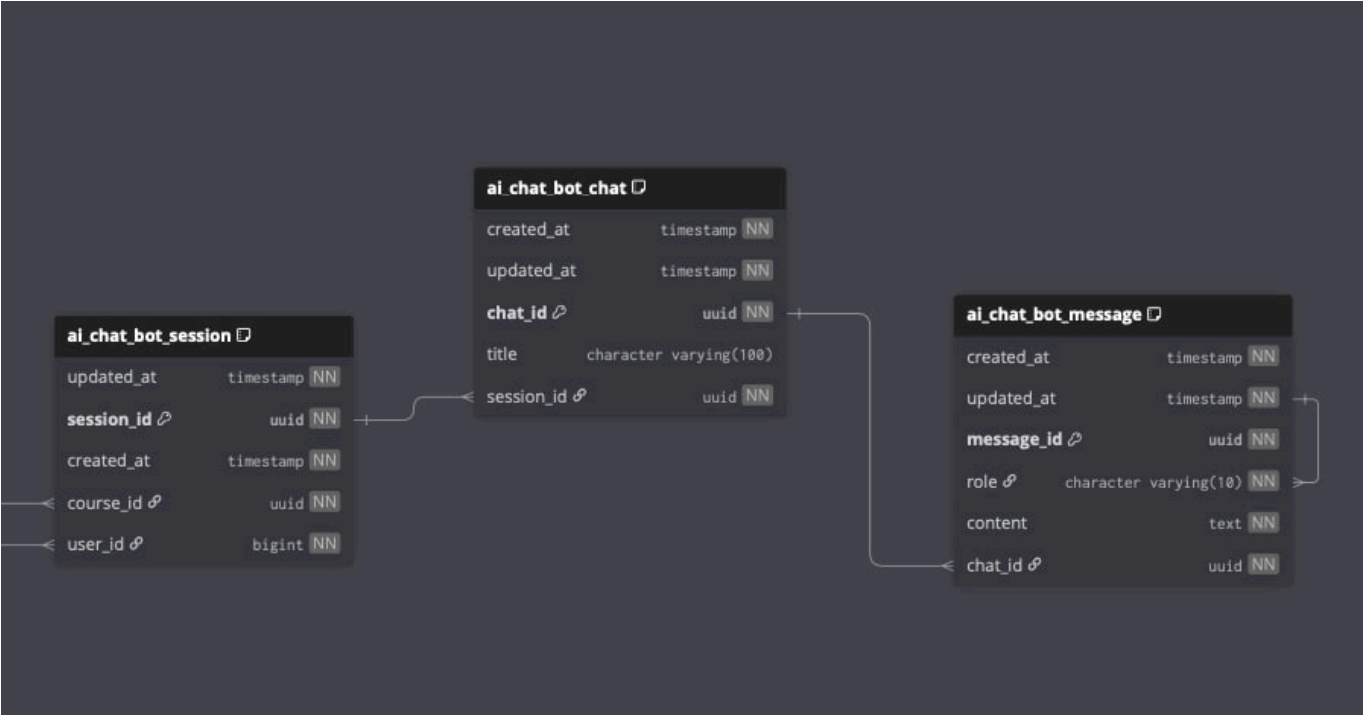


Рисунок П4.7 – Фрагмент ER-диаграммы: сессия ИИ-помощника, чаты и сообщения

Рисунок П4.7 – Кардинальность связей и логика приложения

- 1. Кардинальность: users_user и courses_course -> ai_chat_bot_session (N:1 с каждой стороны в смысле «много сессий у пользователя и у курса»; прикладная пара пользователь-курс индексируется вместе); ai_chat_bot_session -> ai_chat_bot_chat -> ai_chat_bot_message – две степени 1:N.
- 2. Логика: контекст помощника привязан к курсу студента; внутри сессии ведётся несколько чатов, в чате – история ролей user/assistant с ограничением длины текста. Контент курса для RAG синхронизируется во внешнее векторное хранилище (поле yandex_vs_id у курса); ответы стримятся каналом WebSocket в потребителе ai_chat_bot.

Таблицы без отдельного фрагмента ER на рисунках П4.1–П4.7

- 1. admin_panel_invitation: приглашение модератора преподавателю; токен одноразовый, срок жизни и перевод в статусы просрочено/использовано по коду admin_panel; связь users_user (пригласивший) -> приглашения 1:N.
- 2. core_mediaasset / core_assetusage: жизненный цикл файла (pending -> ready, счётчик ссылок, предподписанная загрузка в S3); core_assetusage + django_content_type дают полиморфную связь файла с уроком, задачей, профилем и др. (GenericForeignKey-паттерн Django); удаление неиспользуемых файлов – фоновые задачи core.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. **django_content_type**: служебная таблица фреймворка; в связке с core_assetusage задаёт тип объекта-владельца вложения.

Сводная таблица таблиц базы данных в зоне ответственности

Таблица П4.8 – Таблицы базы данных в зоне ответственности исполнителя

Имя таблицы	Приложение	Назначение
users_user	users	Учётные записи пользователей всех ролей
users_profile	users	Дополнительные данные профиля (аватар, дата рождения)
courses_course	courses	Курсы платформы с метаданными и ценой
courses_course_authors	courses	Связь «многие ко многим» курс–автор
courses_section	courses	Разделы курса с порядковыми номерами
courses_lesson	courses	Уроки с JSON-содержимым
courses_homework	courses	Домашние задания с дедлайном
courses_question	courses	Вопросы с вариантами ответа
courses_task	courses	Задачи с развёрнутым ответом
course_enrollments	courses	Зачисления студентов (через оплату или заявку)
homeworks_attempt	homeworks	Попытки выполнения домашних заданий
homeworks_questionanswer	homeworks	Ответы на вопросы с результатом автопроверки
homeworks_taskanswer	homeworks	Ответы на задачи с развёрнутым ответом
homeworks_taskreview	homeworks	Оценки и комментарии преподавателя к задачам
payments	payments	Платёжные записи с суммой и статусом
payment_items	payments	Позиции платежа (курс и цена)
notifications_notification	notifications	Уведомления всех типов
notifications_notification_read_by	notifications	Записи о прочтении уведомлений
ai_chat_bot_session	ai_chat_bot	Сессии ИИ-помощника (пользователь + курс)
ai_chat_bot_chat	ai_chat_bot	Чаты внутри сессии
ai_chat_bot_message	ai_chat_bot	Сообщения чата
admin_panel_invitation	admin_panel	Приглашения преподавателей с токенами
course_applications	applications	Заявки студентов на специальные курсы
core_mediaasset	core	Метаданные и статус загружаемых файлов
core_assetusage	core	Связи файлов с сущностями системы
django_content_type	django	Реестр моделей (служебная таблица Django)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

[illegible]